Mastering ModusToolbox™ in Your Development Workflow

A Technical Reference for Field Application Engineers

Version: 2.0 — October 2026
ModusToolbox™ Version: 3.9
Target Platform: PSOC Control C3 (KIT_PSC3M6_EVAL)

Table of Contents

1. **Chapter 1: Working Effectively with Version Control & Git**
    - 1.1 The ModusToolbox Project Structure
    - 1.2 The .mtb Dependency Model
    - 1.3 Git: What to Commit, What to Ignore
    - 1.4 The make getlibs Flow
    - 1.5 Git Workflow Best Practices
2. **Chapter 2: Sharing, Importing & Exporting Projects**
    - 2.1 Three Ways to Share a Project
    - 2.2 The mtbarchive Tool
    - 2.3 Git Basics: Committing and Pushing Changes
    - 2.4 Importing a Project into Your IDE
    - 2.5 The Sharing Checklist
3. **Chapter 3: Understanding the ModusToolbox BSP Concept**
    - 3.1 What Is a BSP, Conceptually?
    - 3.2 BSP Directory Structure in Detail
    - 3.3 Device Configurator & design.modus
    - 3.4 BSP Naming, Versioning & Device Families
4. **Chapter 4: Workflows to Develop & Utilize a Custom BSP**
    - 4.1 Why Create a Custom BSP?
    - 4.2 Creating a Custom BSP — Three Workflows
    - 4.3 Key Files to Modify
    - 4.4 Sharing a Custom BSP Across Projects
    - 4.5 Best Practices for Custom BSPs
5. **Chapter 5: Middleware Integration — Library Manager & Makefiles**
    - 5.1 Library Manager Overview
    - 5.2 The mtb_shared Model
    - 5.3 Adding Middleware: What Happens Under the Hood
    - 5.4 Key Makefile Variables
    - 5.5 Finding Middleware Documentation (README & docs/)

- 5.6 COMPONENTS: The Feature Toggle System

6. **Chapter 6: Introduction to the Upcoming ModusToolbox SDK**
   - 6.1 The Current Model vs. the SDK Model
   - 6.2 What the SDK Provides
   - 6.3 Impact on Your Workflow
   - 6.4 Preparing for the SDK Transition
7. **Appendix A: ModusToolbox Command Reference**
8. **Appendix B: Glossary**
9. **Appendix C: Resources**

Chapter 1: Working Effectively with Version Control & Git

ModusToolbox™ projects follow a well-defined structure that separates authored content from generated artifacts. Understanding this structure is essential for effective version control, team collaboration, and troubleshooting build issues. This chapter establishes the foundational knowledge you need to manage ModusToolbox projects professionally using Git.

This session assumes you have completed the out-of-box ModusToolbox training and are familiar with basic project creation and building. We are going deeper into the workflow — the practical knowledge that matters when you are doing real project work with customers.

1.1 The ModusToolbox Project Structure

Every ModusToolbox project follows a predictable directory structure. Understanding this structure is the key to knowing what belongs in version control and what should be regenerated on each machine.

ModusToolbox™ Project Structure

Understanding what to commit and what's generated

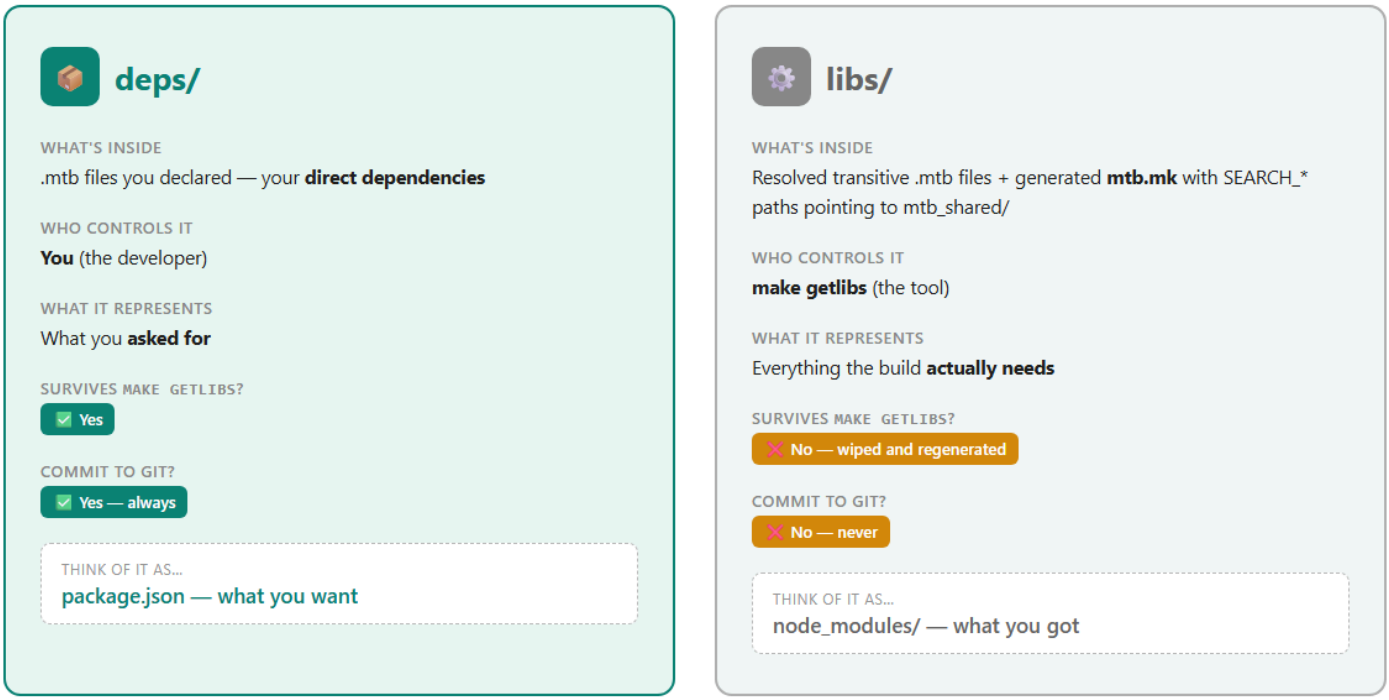
Commit to Git Generated — never commit Shared library cache (one level up)

```
my-project/
├── Makefile                    ← Project build configuration
├── main.c                      ← Your application code
├── deps/                       ← YOUR declared dependencies (committed to Git)
│   └── retarget-io.mtb
├── bsps/                       ← BSP files (local to project)
│   └── TARGET_APP_KIT_PSC3M6_EVAL/
│       ├── cybsp.c / cybsp.h   ← Board init API
│       └── config/
│           └── design.modus    ← Device Configurator master file
├── libs/                       ← Resolved dependency tree (generated, never commit)
│   ├── mtb.mk                  ← Generated SEARCH paths
│   └── (transitive .mtb files + generated .mk files)
├── build/                      ← Build output (never commit)
│
└── ../mtb_shared/              ← SHARED library source (one level up)
    ├── mtb-dsl-psc3m6/
    │   └── release-v1.0.0/
    ├── retarget-io/
    │   └── release-v1.9.0/
    └── ...
```

⚠ **mtb_shared/** lives **one directory level above** the project folder — it is shared across all projects in the workspace.

Understanding deps/ vs libs/

The distinction between `deps/` and `libs/` is the single most important concept for avoiding "it works on my machine" problems. These two directories serve related but fundamentally different purposes.



The Critical Role of libs/mtb.mk

When `make getlibs` resolves your dependency tree, it generates `libs/mtb.mk`. This file is the linchpin that connects your Makefile to the actual library source code. It contains SEARCH_* variables that define include paths for every resolved library:

```makefile
# Example content from libs/mtb.mk (auto-generated)
SEARCH_mtb-dsl-psc3m6=../mtb_shared/mtb-dsl-psc3m6/release-v1.0.0
SEARCH_retarget-io=../mtb_shared/retarget-io/release-v1.9.0
SEARCH_core-lib=../mtb_shared/core-lib/release-v1.7.0
```

The build system automatically includes mtb.mk, which is how your Makefile knows where to find all library source and headers. If mtb.mk is missing or stale (for example, if you skipped `make getlibs` after cloning a project), include paths silently resolve to nothing and builds fail with confusing "missing header" errors.

The mtb_shared/ Directory

A critical architectural detail: **mtb_shared/ lives one level above your project directory**, not inside it. This design enables library sharing across multiple projects in the same workspace.

When you have multiple projects (project-A, project-B, project-C) in a workspace, they can all reference the same physical copy of a library at a specific version. For example, if all three projects use retarget-io version 1.9.0, there is only one copy of that library in mtb_shared/retarget-io/release-v1.9.0/. Each project's libs/mtb.mk points to this shared location.

This architecture keeps disk usage efficient and ensures consistency across projects using the same library version. However, it also means you should never commit mtb_shared/ to Git — it is a workspace-level cache, not a project-specific artifact.

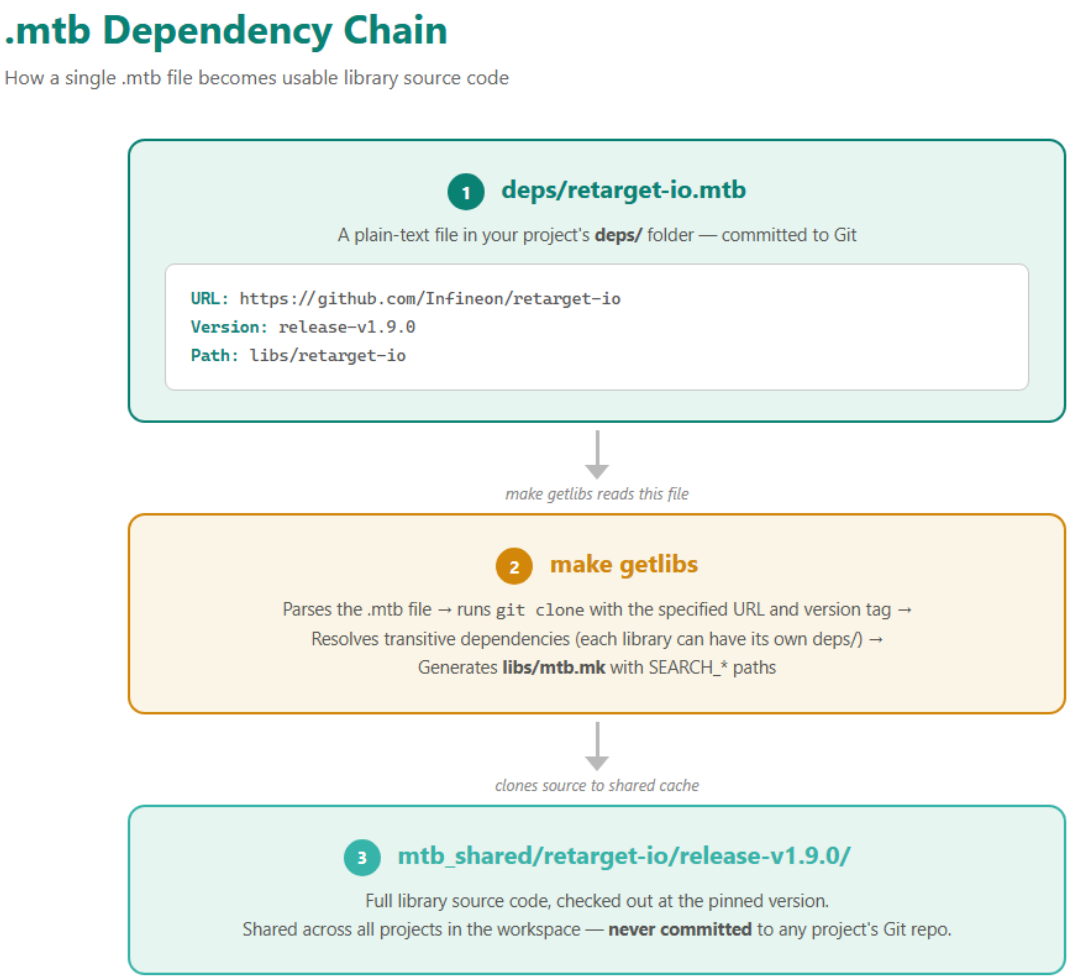
1.2 The .mtb Dependency Model

The .mtb file format is ModusToolbox's mechanism for declaring library dependencies. Each .mtb file is a small text file that points to a Git repository URL and a specific version tag.

```
# This file is auto-generated by Library Manager
https://github.com/cypresssemiconductorco/retarget-io#release-v1.9.0#$$ASSET_REPO$$/retarget-io/release-v1.9.0
```

The file contains three components separated by `#` characters: the Git repository URL, the version tag (typically a release tag like `release-v1.9.0`), and a local path hint that `make getlibs` uses to determine where to clone the library.

When you run `make getlibs`, the tool reads all .mtb files from your project and clones or updates the corresponding libraries into mtb_shared/. The version tag ensures reproducibility — running `make getlibs` on two different machines produces identical library content if the .mtb files are identical.



The .mtb format replaced an older .lib format that you may encounter in legacy projects. The functionality is similar, but .mtb files are the current standard. If you inherit a project with .lib files, ModusToolbox tools will handle them correctly, but new projects should use .mtb exclusively.

The key principle: **.mtb files should be committed to Git**. They are your dependency manifest. When someone clones your project and runs `make getlibs`, the .mtb files tell the tool exactly what to fetch and at what version.

1.3 Git: What to Commit, What to Ignore

Proper Git hygiene for ModusToolbox projects requires understanding which artifacts are authored versus generated. The golden rule is simple: **commit the .mtb files, ignore the mtb_shared/ folder**.

Commit to Git	Ignore (Do Not Commit)
Makefile	build/
main.c and all source files	mtb_shared/ (regenerated by make getlibs)
deps/*.mtb	libs/ (transitive .mtb + mtb.mk pointing to mtb_shared)
bsps/ (BSP customizations)	.vscode/ (user-specific)
.gitignore itself	.cache/ (clangd / IntelliSense index)

Your project's `.gitignore` file should reflect this structure:

gitignore

```
# ModusToolbox build output
build/

# Shared libraries (regenerated by make getlibs)
mtb_shared/
libs/

# IDE-specific
.vscode/
.metadata/
.settings/

# VS Code / clangd IntelliSense cache
.cache/

# Python cache
*.pyc
__pycache__/
```

The rationale for each entry is important to understand:

build/ contains compiled object files, linked binaries, and intermediate build artifacts. These are regenerated by `make build` and should never be committed.

mtb_shared/ is the workspace-level library cache. It may contain libraries from many projects and many versions — far more than any single project needs. Including it in a repository bloats the repo enormously and creates confusion about which libraries the project actually requires.

libs/ is regenerated by `make getlibs`. Committing it would duplicate information already captured in your .mtb files and could cause version mismatches if the committed libs/mtb.mk disagrees with the .mtb declarations.

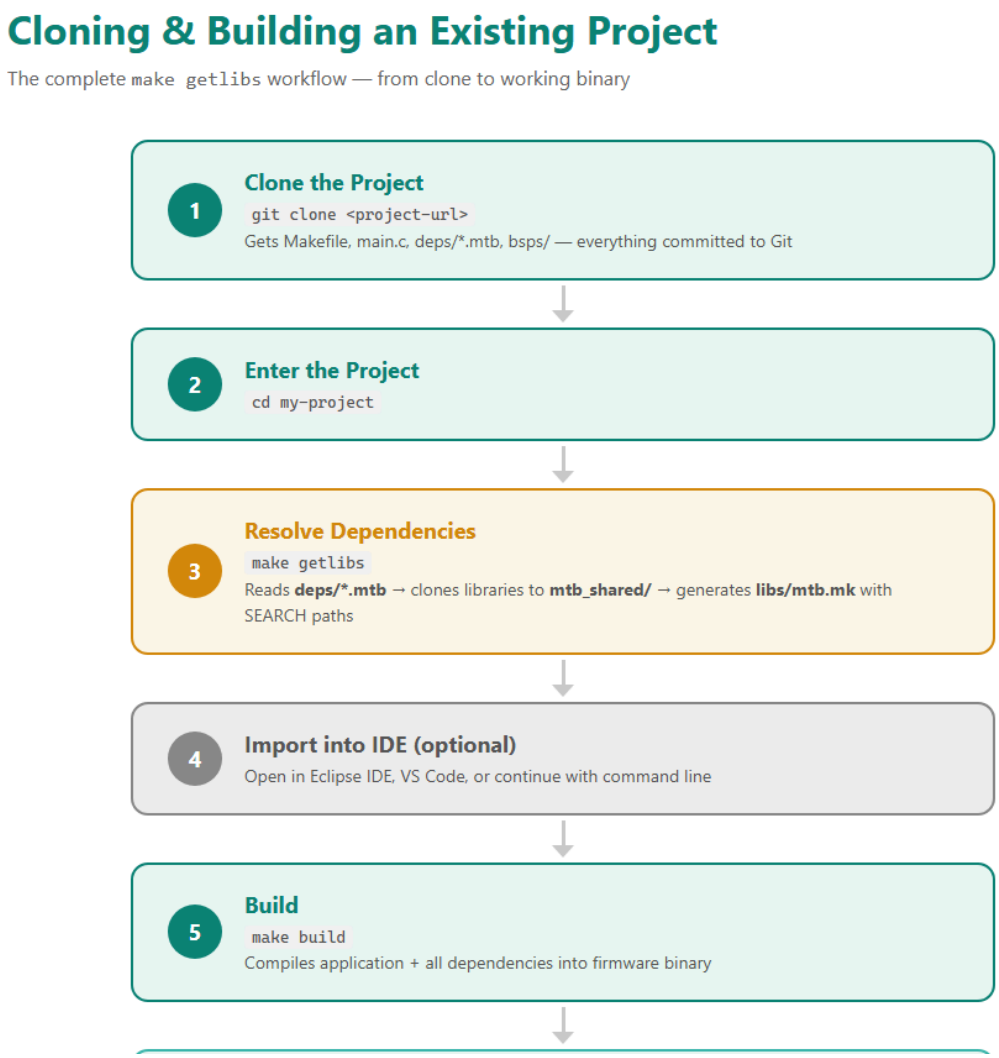
.vscode/ and **.metadata/** contain user-specific IDE settings like window positions, recently opened files, and personal preferences. These vary between developers and should not be shared.

.cache/ contains the clangd or IntelliSense index, which is machine-specific and can be hundreds of megabytes in size.

1.4 The make getlibs Flow

Understanding the `make getlibs` flow is essential for troubleshooting and for onboarding new team members. This section walks through what happens when you clone an existing ModusToolbox project and prepare it for building.

This flow assumes the project was already created (for example, via Project Creator) and committed to a Git repository. This is what happens when you — or a colleague — clone that project on a new machine:



The `make getlibs` command is idempotent — it is safe to run multiple times. If libraries already exist at the correct version in mtb_shared/, they are not re-downloaded. If a library version has changed in your .mtb files, the new version is fetched and libs/mtb.mk is regenerated.

IDE Import Options

There are two general approaches to getting a cloned project into your IDE (command-line and IDE GUI):

Approach	How	make getlibs required first?
Command-line export	`make eclipse` or `make vscode`	Yes — run `make getlibs` yourself, then generate IDE files
IDE import (MTB-specific)	Eclipse: Import ModusToolbox Application In-Place; VS Code: via MTB Assistant extension	Likely handled automatically during import
IAR / µVision	See respective export sections in the ModusToolbox User Guide	Refer to MTB User Guide for details

The command-line approach gives you full control: you run `make getlibs` explicitly, then generate IDE project files. The IDE's built-in ModusToolbox import typically handles dependency resolution automatically, making it the easier path for most users. Either way, the result is the same: a fully resolved project ready to build and debug.

1.5 Git Workflow Best Practices

Effective Git usage with ModusToolbox projects requires a few additional considerations beyond standard Git best practices.

Practice	Why
One project per repo (preferred)	Clean history, independent versioning
Commit .mtb files	Reproducible dependency resolution
Never commit mtb_shared/	It is regenerated; committing bloats the repo enormously
Tag releases	Match your project version to tested BSP/middleware versions
Document BSP + MTB version in README	e.g., "Tested with KIT_PSC3M6_EVAL BSP, ModusToolbox 3.9"
Use branches for BSP variants	If supporting multiple boards, branch per target

The **one project per repo** recommendation keeps your Git history clean and allows independent versioning of each project. While ModusToolbox workspaces can contain multiple projects, each project should ideally have its own Git repository to avoid coupling unrelated changes.

Tagging releases is particularly important for ModusToolbox projects. A Git tag represents a known-good combination of your application code, BSP version, and middleware versions. When you release firmware version 1.2.0, the tag captures the exact state of all dependencies at that moment.

Documenting versions in your README provides an immediate reference for anyone cloning the project. Include the ModusToolbox version used to create and test the project, the BSP name and version, and any critical middleware versions. This documentation is invaluable for troubleshooting version mismatches.

Chapter 2: Sharing, Importing & Exporting Projects

Moving ModusToolbox™ projects between developers, machines, and organizations requires understanding the various sharing mechanisms available. This chapter covers the practical workflows for project sharing, from simple Git operations to the specialized mtbarchive tool.

2.1 Three Ways to Share a Project

ModusToolbox projects can be shared using three primary methods, each suited to different scenarios:

Scenario	Method	What controls the outcome	Includes mtb_shared?
Git repo (push/pull)	Cleanest for teams with MTB installed	.gitignore — defines what is tracked vs. excluded	No — recipient runs make getlibs
mtbarchive (GUI or CLI)	MTB-aware archive — automatically includes the right files, excludes generated artifacts	Built-in best practices — tool knows the MTB project structure	No — recipient runs make getlibs
IDE Export / manual ZIP	IDE "Export ModusToolbox Application" or hand-curated ZIP	User's understanding of what to include — YMMV	No — see warning

Git repositories are the cleanest approach for technical teams where all members have ModusToolbox installed. The repository contains only authored content (source code, Makefile, .mtb files, BSP configuration), and each developer runs `make getlibs` to fetch dependencies. This approach produces the smallest repository size and ensures everyone builds from the same dependency declarations.

mtbarchive is a ModusToolbox-aware archiving tool that creates self-contained project ZIPs with intelligent include/exclude logic. It is ideal for sharing with customers, attaching to support tickets, or transferring projects via email or USB. The tool understands the ModusToolbox project structure and automatically includes the right files while excluding generated artifacts.

IDE Export or manual ZIP puts you in full control, which also means full responsibility. Results vary based on the user's understanding of what should and should not be included. This approach is the fallback when the other methods are not available.

Critical Warning: Do not include mtb_shared/ in archives unless there is a specific reason. The mtb_shared/ directory is a workspace-level cache shared across all projects — it contains every library version you have ever resolved, not just the ones your project uses. Including it ships unnecessary files (often 200+ MB), creates confusion for the recipient, and may be overwritten anyway if the import process re-runs `make getlibs`. Always let the recipient run `make getlibs` to get exactly what the project needs.

2.2 The mtbarchive Tool

The mtbarchive tool is ModusToolbox's purpose-built solution for creating shareable project archives. It is available as a graphical interface (mtbarchive-gui.exe) and a command-line tool (mtbarchive.exe).

What mtbarchive Includes vs. Excludes

Included	Excluded
Source code, Makefile, configs	libs/ (regenerated by make getlibs)
deps/ (.mtb dependency declarations)	build/ output
bsps/ (BSP configuration & device config)	mtb_shared/ (regenerated)
.git/ history (if present)	IDE workspace files
README, LICENSE	

The tool understands which artifacts are generated versus authored, so you do not have to manually curate the archive contents. This eliminates the common mistake of accidentally including hundreds of megabytes of library code.

GUI Access (Recommended)

The graphical interface can be launched from multiple locations:

- **Eclipse IDE**: Quick Panel → "ModusToolbox archiving and sharing" (under Tools)
- **VS Code**: ModusToolbox Assistant extension → Application tab
- **Direct execution**: Run mtbarchive-gui.exe from the ModusToolbox installation

The GUI provides visual file selection and the same intelligent include/exclude logic as the command line.

Command-Line Usage

The `mtbarchive` CLI is not on the PATH by default in Modus-Shell or the IDE terminal. To use it, specify the full path:

```
# Basic usage (full path required):
C:/Users/<user>/ModusToolbox/tools_3.9/mtbarchive/mtbarchive.exe -o my_project_archive.zip -f <project-folder>
```

```
# Verbose mode (shows what is being packaged):
C:/Users/<user>/ModusToolbox/tools_3.9/mtbarchive/mtbarchive.exe -v -o my_project_archive.zip -f <project-folder>
```

The `-v` (verbose) flag is useful for verifying that the archive contains what you expect. It lists each file as it is added to the archive.

Why Use mtbarchive Over a Plain ZIP?

A plain ZIP created manually risks including generated artifacts (build/, libs/, mtb_shared/) that bloat the archive and confuse the recipient. The mtbarchive tool enforces best practices automatically:

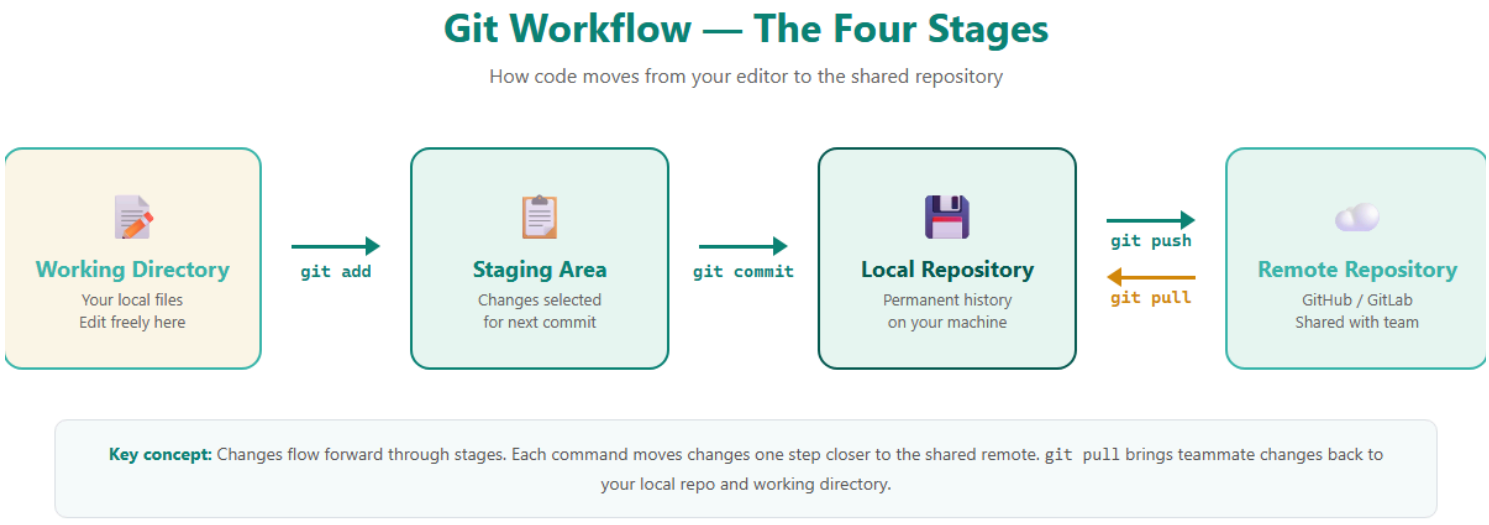
- It knows which artifacts are generated versus authored
- It produces a clean archive that recipients can open with `make getlibs` → `make build`
- It is ideal for support tickets, customer handoff, and email/USB sharing

Think of mtbarchive as `git archive` but MTB-aware — it understands the project structure so you do not have to manually curate what goes in.

2.3 Git Basics: Committing and Pushing Changes

For FAEs who may not use Git daily, this section provides a quick refresher on the essential Git workflow.

The Git workflow moves changes through four stages:



The staging area concept is key: `git add` selects what goes into the next commit, `git commit` saves it locally, and `git push` shares it with the remote repository.

Key Commands

```
# Check which files have changed
git status

# Stage a specific file
git add <file>

# Stage all changed files
git add .

# Commit staged changes with a message
git commit -m "Add UART debug output to main loop"

# Push commits to the remote repository
git push origin main

# Pull changes from the remote repository
git pull origin main
```

Best Practices for ModusToolbox Projects

Practice	Why
Commit often, in small logical chunks	Easier to review, revert, and bisect issues
Write descriptive commit messages	Future-you (and teammates) will thank you
Always `make build` before committing	Do not push broken code
Review `git status` before committing	Catch unintended files (e.g., build/ artifacts)

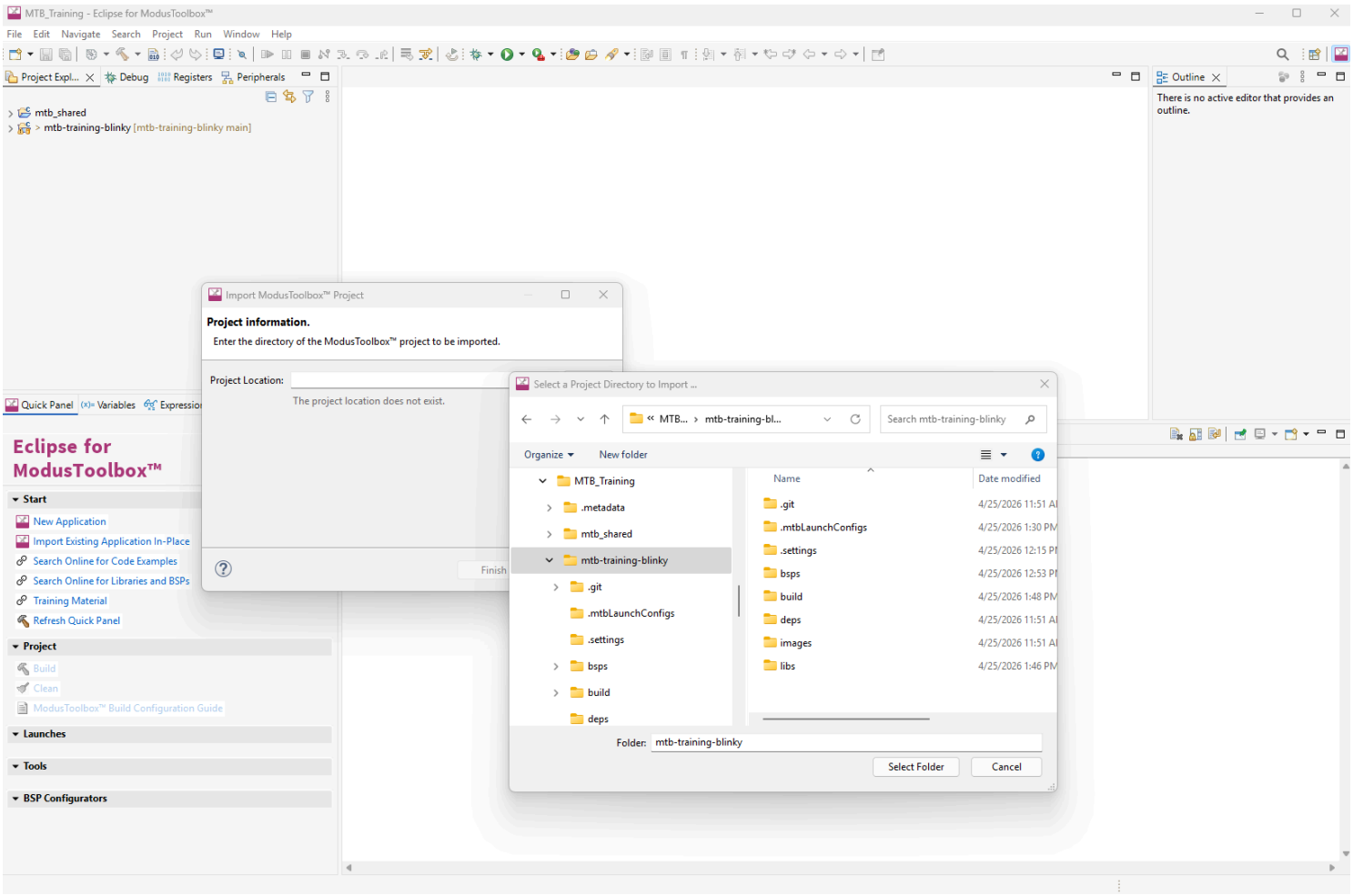
The most common mistake is committing without checking `git status` first. This can lead to accidentally tracking generated files that should be in .gitignore. Always verify what you are about to commit.

2.4 Importing a Project into Your IDE

After cloning or receiving a ModusToolbox project, you need to import it into your development environment. The process varies by IDE.

Eclipse IDE

1. Clone or copy the project into your desired workspace folder
2. Launch Eclipse IDE and set the workspace to the project's parent folder
3. From the Quick Panel, click **Import ModusToolbox Application In-Place**
4. Select the project folder — Eclipse registers it as a managed MTB project
5. Right-click the project → **Properties** → **ModusToolbox** → select/verify the installed ModusToolbox version

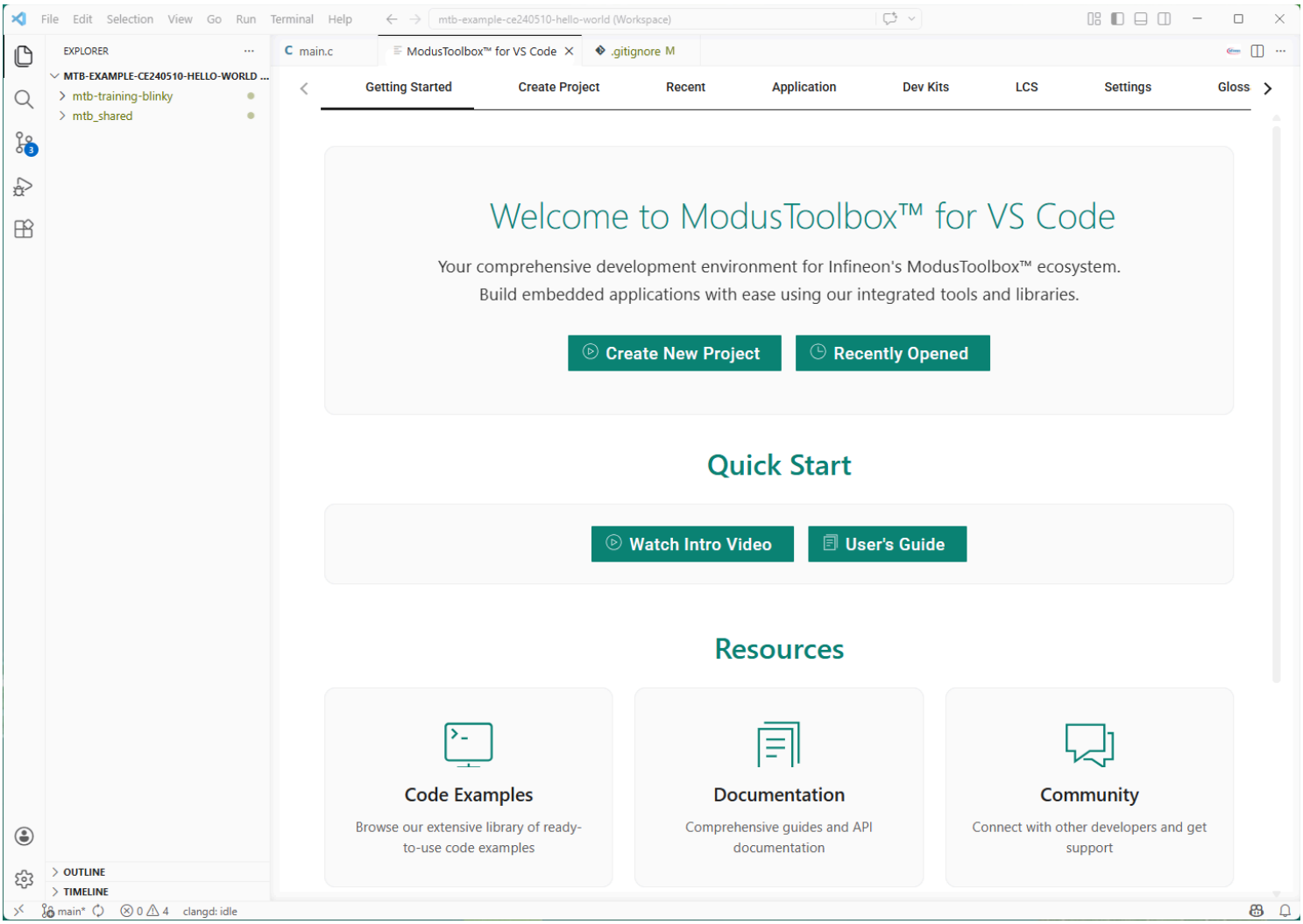


Important: The workspace path matters. Eclipse expects projects to be direct children of the workspace folder. If your project is nested more deeply, Eclipse may not recognize it correctly.

> ⚠️ **Eclipse 2026.1+ requirement**: Projects must be explicitly associated with an installed ModusToolbox version via Project Properties. If this step is skipped, builds and tool launches may fail or use an unexpected version.

VS Code (with ModusToolbox Extension — Recommended)

1. Clone or copy the project to the desired location
2. Open VS Code → File → Open Folder... → select the project folder
3. The ModusToolbox extension automatically detects the project and provides IntelliSense, build tasks, and debug configurations



The ModusToolbox Assistant extension handles most setup automatically, including dependency resolution and build system integration.

VS Code (without ModusToolbox Extension)

If you are not using the ModusToolbox extension, you need an additional step:

1. Clone or copy the project to the desired location
2. Open Modus-Shell, navigate to the project folder
3. Run `make getlibs` — this clones the required libraries and restored content as specified in the projects /deps and bsp folder
4. Run `make vscode` — this generates .vscode/ settings and a .code-workspace file
5. Open the generated .code-workspace file in VS Code

The generated workspace file contains the necessary settings for building and debugging the project.

IAR / µVision

For IAR Embedded Workbench and Arm Keil µVision, use the corresponding make targets to generate IDE project files:

```
# For IAR Embedded Workbench
make ewarm8

# For Keil µVision
make uvision5
```

Refer to the ModusToolbox User Guide for detailed instructions on these workflows.

Common Pitfall: Version Mismatches

If the recipient has a different ModusToolbox version than the project author, BSP or middleware versions may conflict. Always document your MTB version in the project README to help recipients identify potential compatibility issues.

2.5 The Sharing Checklist

Before sharing a ModusToolbox project, verify that it meets these criteria:

Item	✓
README documents: MTB version, BSP, target board, tested behavior	
.mtb files present in deps/	
.gitignore excludes build/, mtb_shared/, libs/	
Project builds clean from `make getlibs` + `make build` (test it!)	
No absolute paths in Makefile or configurations	
License file included (if sharing externally)	

The most important test is the last point about absolute paths. A project that works on your machine but contains hardcoded paths like `C:\Users\YourName\...` will fail on any other machine. Use relative paths throughout your Makefile and configuration files.

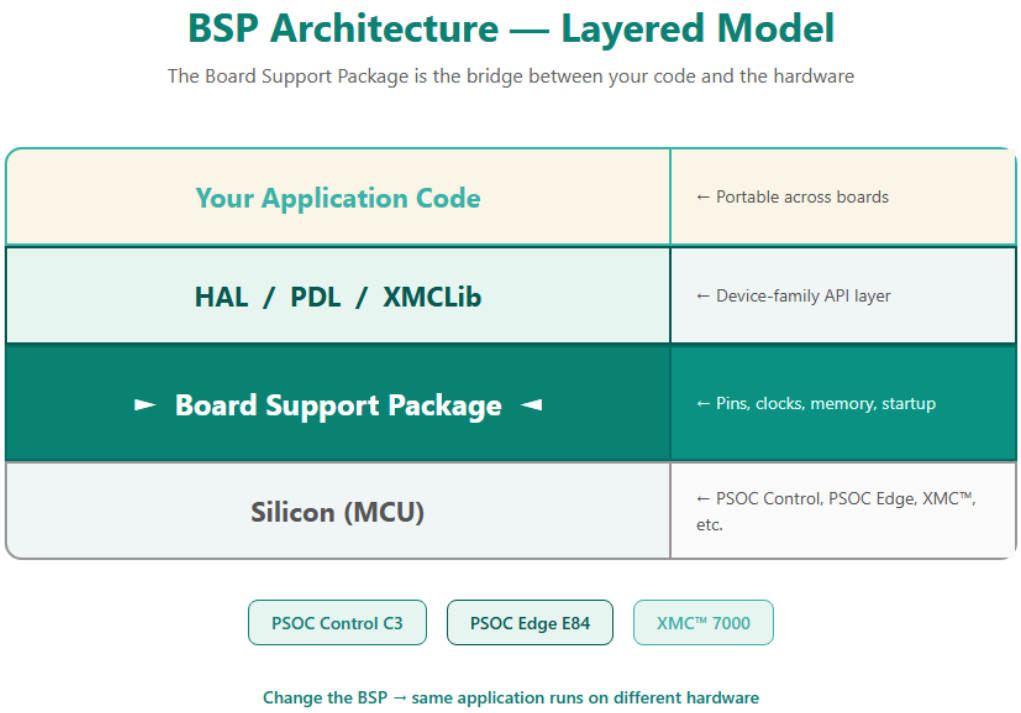
Testing the full clone-to-build workflow before sharing catches most issues: delete your mtb_shared/ and libs/ directories, run `make getlibs`, then `make build`. If it succeeds, the project is likely ready to share.

Chapter 3: Understanding the ModusToolbox BSP Concept

The Board Support Package (BSP) is the foundation that allows your application code to run on specific hardware. This chapter explains what a BSP is, how it is structured, and how the Device Configurator generates the configuration code that makes `cybsp_init()` work.

3.1 What Is a BSP, Conceptually?

A BSP sits between your application code and the silicon, providing the hardware-specific initialization and configuration that makes portable application code possible.



The BSP answers three fundamental questions about your hardware:

1. **Which chip?** — Device ID, core type (Cortex-M33, Cortex-M55, etc.), flash size, RAM size
2. **How is it wired?** — Pin assignments for peripherals, crystal frequencies, debug interface configuration
3. **How does it boot?** — Startup code, linker scripts, memory map configuration

When your application calls `cybsp_init()`, it is invoking the BSP's board initialization function. This function configures the clock tree, sets up pin multiplexing, initializes power management, and prepares all the hardware so your application can use HAL and PDL functions without worrying about low-level hardware details.

The BSP abstraction allows you to write application code that is largely portable across different boards. If you change from one eval kit to a custom production board, you create or modify a BSP rather than scattering hardware-specific code throughout your application.

3.2 BSP Directory Structure in Detail

Understanding the BSP directory structure helps you navigate configuration files, troubleshoot initialization issues, and customize the BSP for custom hardware.

BSP Directory Structure

Inside TARGET_APP_KIT_PSC3M6_EVAL — what each file does

Configuration files | Auto-generated source | Toolchain-specific | Core BSP files

```
bsps/TARGET_APP_KIT_PSC3M6_EVAL/
├── config/
│   ├── design.modus                ← Device Configurator master file
│   └── GeneratedSource/            ← Auto-generated from design.modus
│       ├── cycfg.h / cycfg.c
│       ├── cycfg_pins.h / .c       ← Pin assignments
│       ├── cycfg_clocks.h / .c     ← Clock configuration
│       ├── cycfg_peripherals.h / .c ← UART, SPI, I2C, Timer
│       └── cycfg_system.h / .c     ← Power modes, WDT
├── cybsp.h / cybsp.c               ← Board init API — cybsp_init()
├── system_*.h                      ← System header
├── linker/
│   ├── TOOLCHAIN_GCC_ARM/          ← GCC linker scripts
│   ├── TOOLCHAIN_ARM/              ← ARM Compiler linker scripts
│   └── TOOLCHAIN_IAR/              ← IAR linker scripts
├── bsp.mk                          ← BSP build configuration
└── deps/                           ← BSP's own dependencies
```

Key Files and Directories

config/design.modus is the master configuration file for the Device Configurator. This binary XML file stores all your pin assignments, clock tree configuration, and peripheral settings. You should never edit this file manually — always use the Device Configurator GUI.

config/GeneratedSource/ contains the C source files generated from design.modus. These files are regenerated automatically when you save changes in Device Configurator. The generated files follow a consistent naming pattern:

- `cycfg.c/.h` — Top-level configuration entry point
- `cycfg_pins.c/.h` — GPIO and pin multiplexing
- `cycfg_clocks.c/.h` — Clock tree (PLL, FLL, HF clocks)
- `cycfg_peripherals.c/.h` — UART, SPI, I2C, Timer configurations
- `cycfg_system.c/.h` — Power modes, watchdog timer, system configuration

cybsp.h / cybsp.c define the board initialization API, including the `cybsp_init()` function and board-level macros like `CYBSP_USER_LED` and `CYBSP_DEBUG_UART_TX`. These files are auto-generated by Device Configurator from `design.modus` — do not edit them manually.

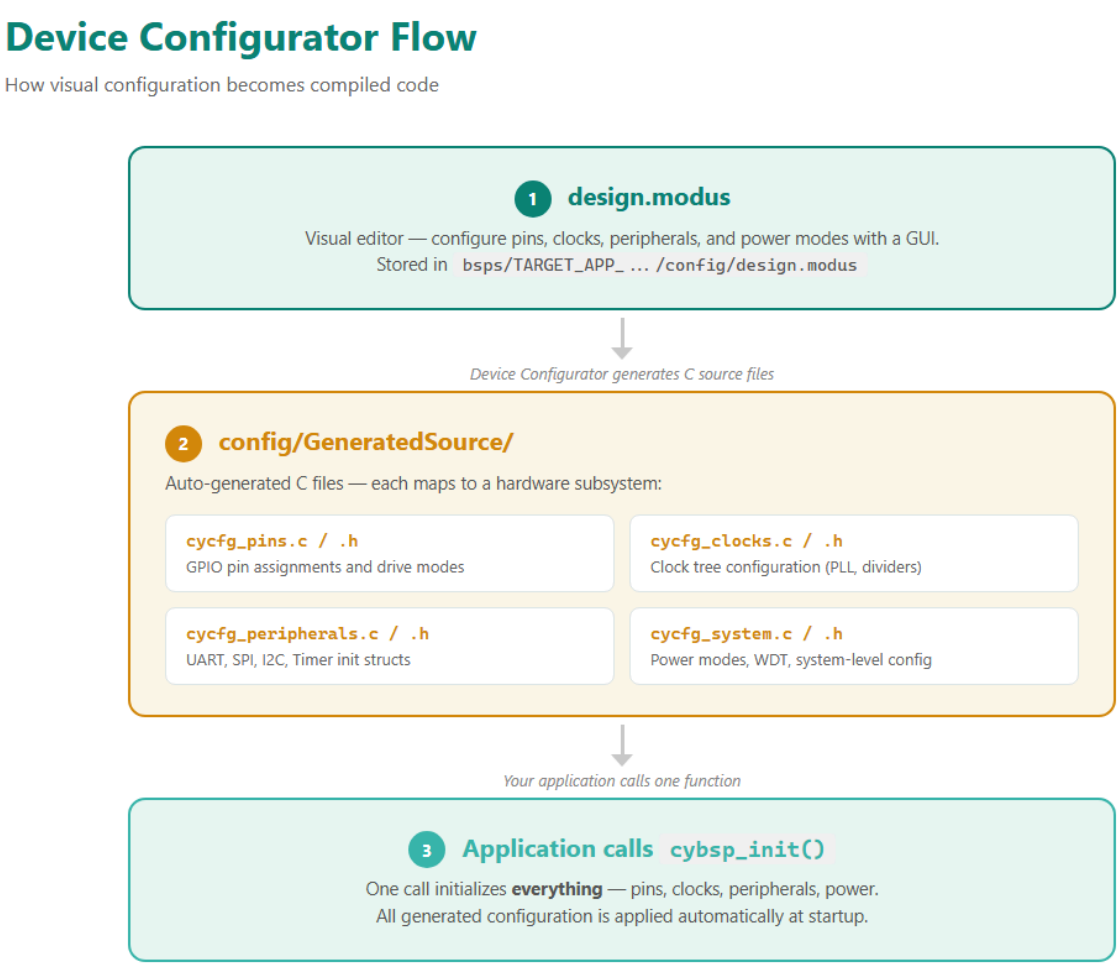
linker/ contains linker scripts for each supported toolchain (GCC, ARM Compiler, IAR). These scripts define the memory map — where code, data, and stack are placed in flash and RAM.

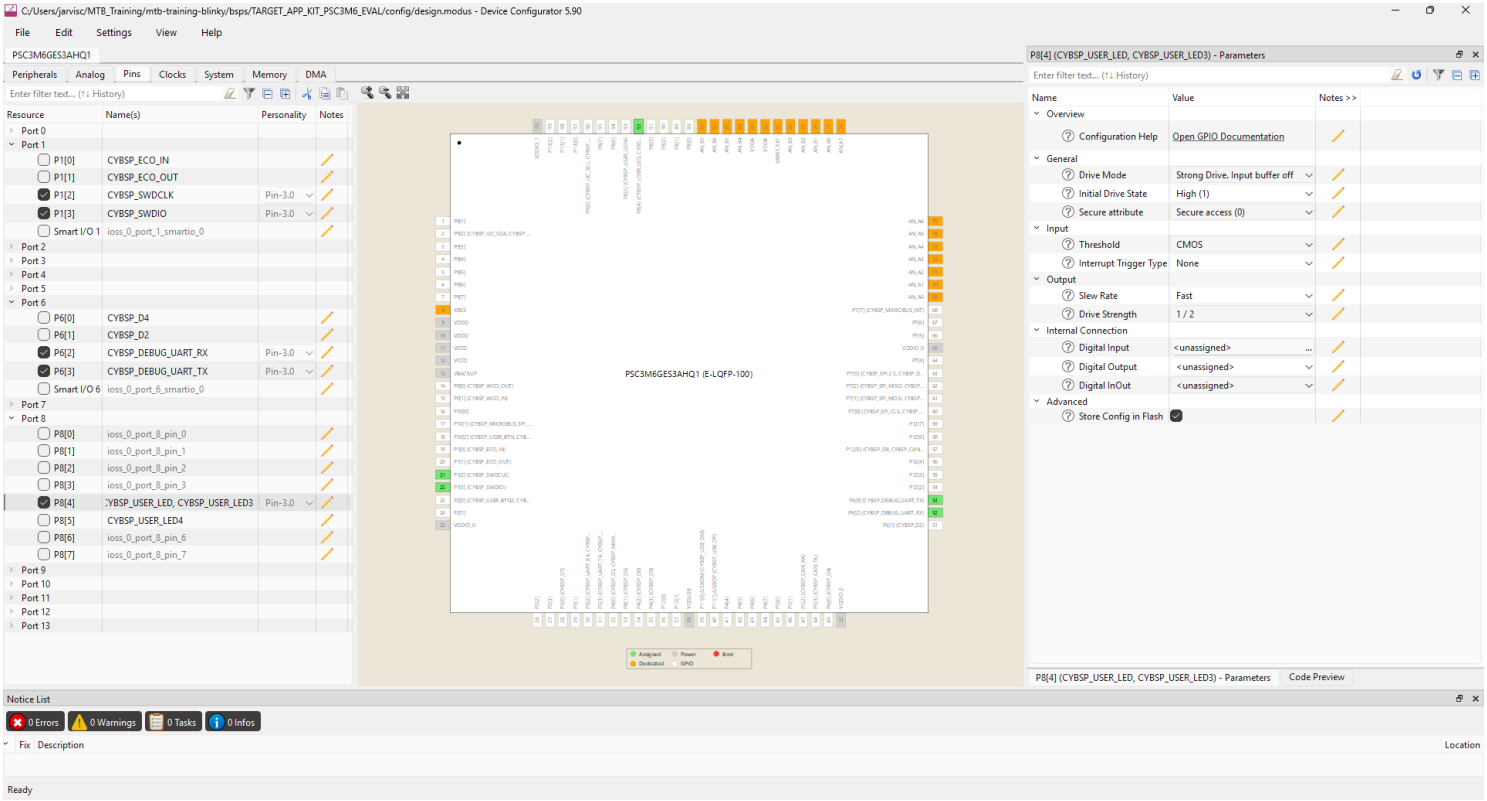
bsp.mk is a makefile that defines BSP-level build settings. This includes the DEVICE variable (the silicon part number), COMPONENTS that should be enabled by default, and any feature flags. You will rarely edit this file directly, but it is useful for troubleshooting when you need to understand why a specific define or component is active.

deps/ within the BSP contains the BSP's own dependencies. On PSC3 family devices (including PSC3M6), this is a single consolidated **Device Support Library (DSL)** rather than separate HAL and PDL dependencies — see the glossary entry for DSL.

3.3 Device Configurator & design.modus

The Device Configurator is a visual tool that generates the low-level initialization code for your BSP. It provides a graphical interface for configuring pins, clocks, and peripherals without writing initialization code by hand.





Each generated file handles a specific configuration domain:

Generated File	Configuration
cycfg_pins.c	GPIO assignments, pin multiplexing
cycfg_clocks.c	Clock tree (PLL, FLL, HF clocks, dividers)
cycfg_peripherals.c	UART, SPI, I2C, Timer configurations
cycfg_system.c	Power modes, watchdog timer, etc.

Important: design.modus is a binary XML file. Always edit it through the Device Configurator GUI, never with a text editor. Manual edits can corrupt the file format and cause Device Configurator to fail.

When you switch to a different BSP (for example, moving from a PSOC Control C3 eval kit to a PSOC 6 eval kit), you get a completely different design.modus file configured for that board's hardware.

3.4 BSP Naming, Versioning & Device Families

BSP names follow specific conventions that indicate their origin, purpose, and target hardware.

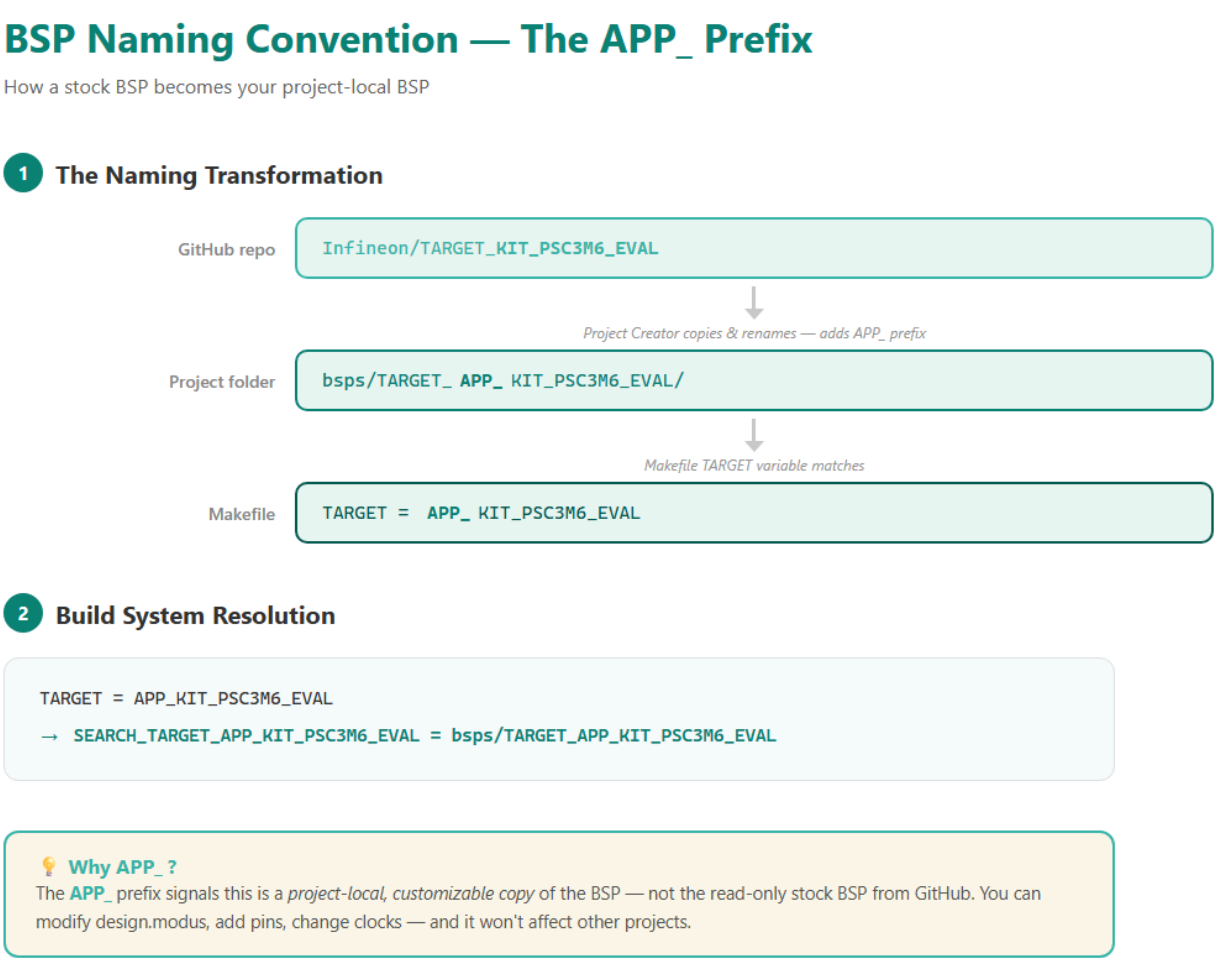
TARGET Naming Pattern

All BSP directories follow the pattern `TARGET_<BSP_NAME>`. The TARGET_ prefix is required by the build system.

Infineon Naming Conventions

Prefix	Meaning	Example
KIT_	Official Infineon eval kit	TARGET_KIT_PSC3M6_EVAL

Prefix	Meaning	Example
CY8CKIT-	Legacy PSOC 6 kit naming	TARGET_CY8CKIT-062-BLE
CY8CPROTO-	Legacy PSOC 6 proto board	TARGET_CY8CPROTO-062-4343W
Custom	User-defined for custom boards	TARGET_MY_CUSTOM_BOARD



The APP_ Prefix Convention

When you create a project using Project Creator, it copies the kit BSP into your project's bsps/ folder and automatically prepends `APP_` to the target name:

```
GitHub repo:      Infineon/TARGET_KIT_PSC3M6_EVAL        (stock BSP source)
                          ↓ Project Creator copies & renames
Project folder:   bsps/TARGET_APP_KIT_PSC3M6_EVAL/       (project-local copy)
Makefile:         TARGET=APP_KIT_PSC3M6_EVAL
```

The APP_ prefix signals that this is a project-local copy that can be customized without affecting the original kit BSP. This is an important distinction: the BSP in your project is yours to modify. The original kit BSP in the library cache remains untouched.

How the Build System Resolves the BSP

The build system uses the TARGET variable in your Makefile to locate the BSP:

```makefile
# In Makefile:
TARGET=APP_KIT_PSC3M6_EVAL

# In libs/mtb.mk (generated by make getlibs):
SEARCH_TARGET_APP_KIT_PSC3M6_EVAL=bsps/TARGET_APP_KIT_PSC3M6_EVAL
```

For custom BSPs, you do not need the APP_ prefix — just follow the TARGET_ folder convention:

```
Folder:    bsps/TARGET_MY_CUSTOM_BOARD/
Makefile:  TARGET=MY_CUSTOM_BOARD
```

Device Family Categories

Infineon's MCU families are organized into categories that determine which "recipe" makefile the build system uses:

Category	Recipe	MCU Families
cat1	recipe-make-cat1	PSOC 6, PSOC 4
cat1b	recipe-make-cat1b	PSOC Control C3
cat1d	recipe-make-cat1d	PSOC Edge (E84)
cat3	recipe-make-cat3	XMC™ 1000/4000

The recipe determines toolchain settings, memory layout conventions, and other build behaviors specific to each device family.

Chapter 4: Workflows to Develop & Utilize a Custom BSP

Most customer projects eventually require a custom BSP tailored to production hardware. This chapter covers when to create a custom BSP, the three workflows available, and best practices for BSP development and sharing.

4.1 Why Create a Custom BSP?

The kit BSP provided with ModusToolbox™ is designed for Infineon's evaluation boards. While it is perfectly suitable for prototyping and getting started, production hardware typically differs in significant ways:

Scenario	What Changes
Extending a kit BSP	Enable additional peripherals, change clock speeds, reconfigure pins to use more of the device's capabilities
Customer production board	Different pin assignments, no debug header, custom crystal, different memory layout
Multi-project sharing	Common BSP across a product line, maintained independently from application code

Key Principle: When targeting custom hardware, create a dedicated BSP rather than continually patching the kit BSP. This gives you a clean baseline for the customer's specific board and makes it easier to track changes from the original design.

The kit BSP is absolutely expected to be modified for prototyping and evaluation — that is why Project Creator copies it into your project. But when customers move to their own hardware, guide them to create a dedicated BSP that clearly represents their board.

There are two general paths to creating a custom BSP:

Approach	When to Use
BSP Assistant → "Create New BSP based on..."	Customer based their schematic on an Infineon reference board — start from that kit BSP as a foundation (recommended)
Manual copy & rename	Full control — copy bsps/TARGET_APP_KIT_*/ to bsps/TARGET_MY_BOARD/, update Makefile TARGET=

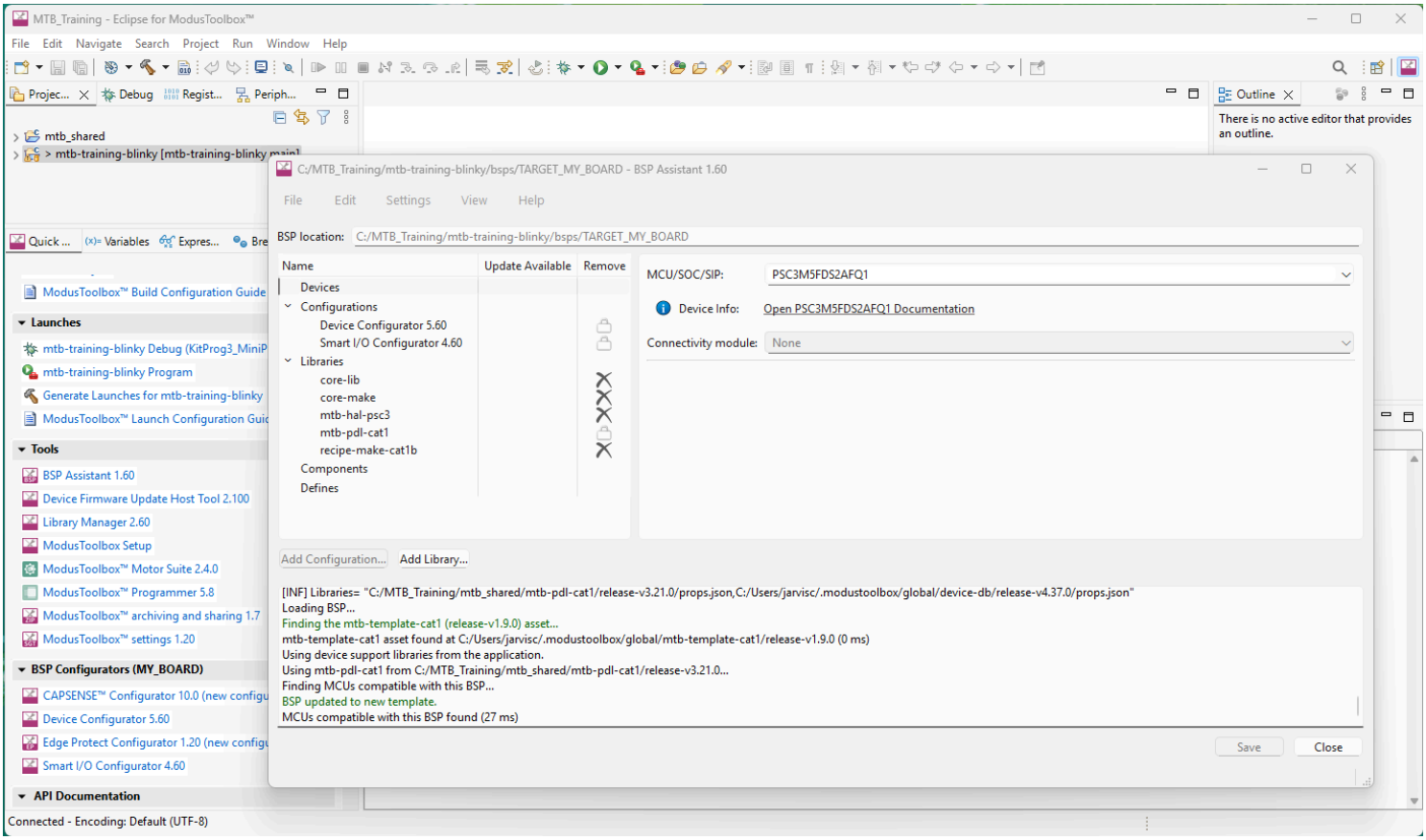
4.2 Creating a Custom BSP — Three Workflows

ModusToolbox provides three workflows for creating or modifying BSPs, each suited to different scenarios.

Option A: Modify the Current BSP In-Place

This workflow is the quickest when you want to customize the BSP that is already in your project.

1. (Optional) Copy the current BSP first if you want to preserve the original
2. Open BSP Assistant from Eclipse Quick Panel or VS Code ModusToolbox Assistant extension — this modifies the CURRENT BSP
3. Change device (MPN), pins, clocks, peripherals as needed
4. Save — Device Configurator regenerates configuration
5. Build and test



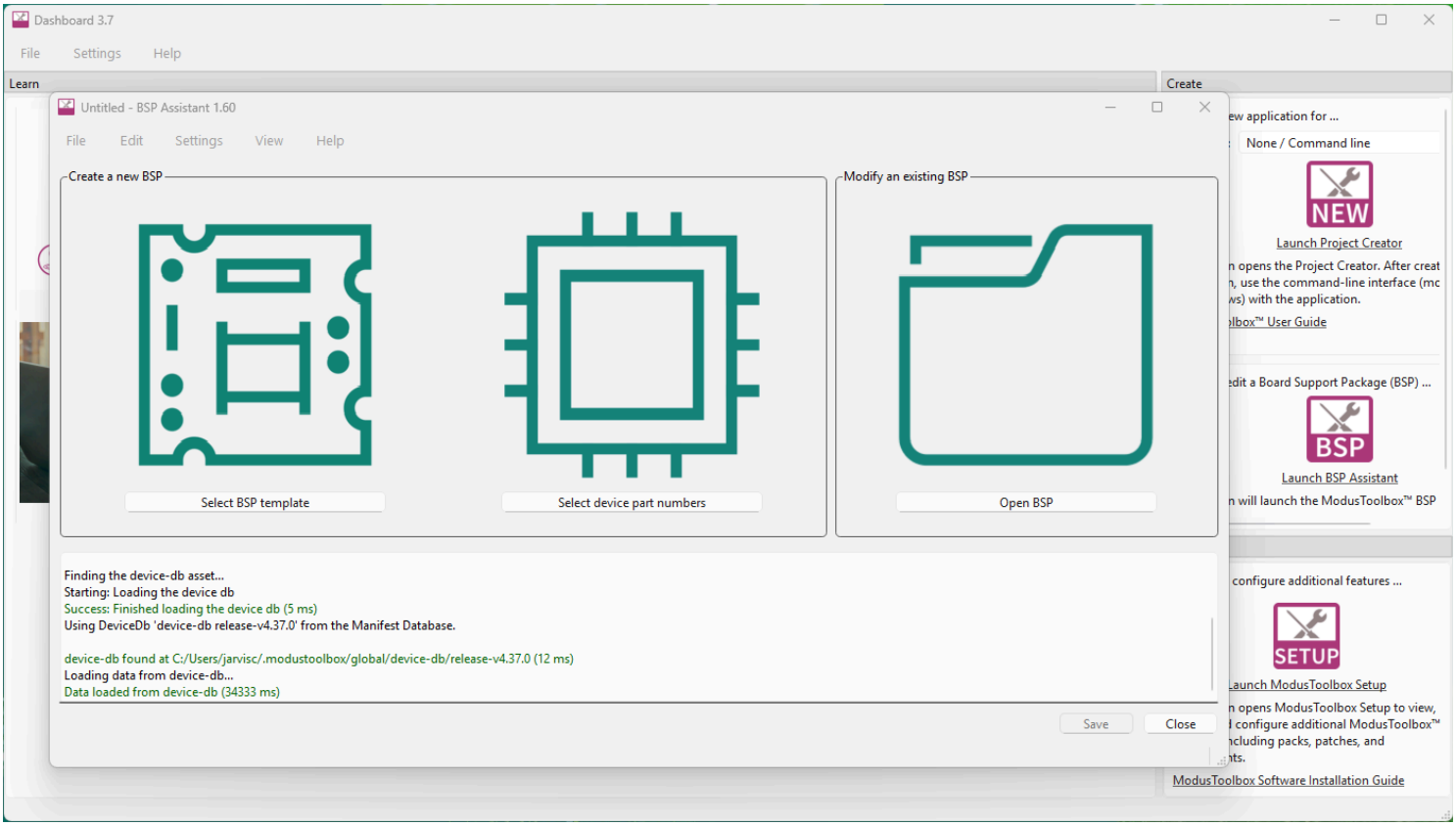
Warning: When launched from within a project (Eclipse or VS Code), BSP Assistant edits the active BSP in place. If you want to keep the original kit BSP untouched, copy it to a new folder first. Many developers prefer to maintain both the original APP_KIT_* BSP and a custom

TARGET_MY_BOARD/ BSP so they can compare configurations.

Option B: Create a New BSP Externally

This workflow creates a fresh BSP based on an existing Infineon kit BSP, keeping the original untouched.

1. Launch BSP Assistant from the ModusToolbox Dashboard (or Windows Start menu) — select "Create New BSP based on..." an existing Infineon kit BSP
2. Save the new BSP to either:
   - Directly into the project's bsps/ folder, or
   - An external location for sharing across projects
3. Open Library Manager in the project: select and enable the new BSP (this updates the Makefile `TARGET=` and runs `make getlibs` automatically)
4. Edit design.modus as needed (cybsp.h is auto-generated when you save)
5. Build and test



Important: BSP Assistant is the only supported way to change the target device (MPN). If you need to switch from a PSOC Control C3M6 to a PSOC Control C3M7, use BSP Assistant — do not attempt to manually edit device identifiers.

Option C: Manual Copy & Rename

This workflow provides maximum control and is useful for understanding the BSP structure or for troubleshooting.

1. Copy `bsps/TARGET_APP_KIT_PSC3M6_EVAL/` to `bsps/TARGET_MY_CUSTOM_BOARD/`
2. Update Makefile: `TARGET=MY_CUSTOM_BOARD`
3. Run `make getlibs` to regenerate `libs/mtb.mk` with the new BSP path
4. Edit design.modus as needed (cybsp.h regenerates automatically)

Warning: Manual copy cannot change the target device (MPN). If you copy a PSOC Control C3M6 BSP and try to use it with a different device variant, the configuration may be incorrect. Only BSP Assistant supports MPN changes.

4.3 Key Files to Modify

When customizing a BSP for production hardware, these are the files you will typically modify:

File	What to Modify	Tool
config/design.modus	Pin assignments, clock tree, peripheral config	Device Configurator
cybsp.h	Board-level macros (CYBSP_USER_LED, CYBSP_DEBUG_UART, etc.)	Auto-generated by Device Configurator
Makefile	DEVICE variable (if different silicon variant)	Text editor
linker/*.ld	Memory regions (if custom flash/RAM layout)	Text editor (advanced)
COMPONENT_*/	Core-specific resources (dual-core only)	Depends on change

What NOT to Modify

- **system_*.c**: These files contain low-level startup code generated for the specific device. Do not modify unless you fully understand the implications.
- **GeneratedSource/**: Do not manually edit these files — they are regenerated from design.modus every time you save in Device Configurator. Your changes will be overwritten.

4.4 Sharing a Custom BSP Across Projects

Once you have created a custom BSP, you may want to share it across multiple projects or with team members. Three approaches are available:

Method	How	Best For
Project-local	BSP lives in bsps/ inside the project	Single project, self-contained
Shared via Git repo	BSP(s) committed to a dedicated repo, cloned locally	Team-wide use, multiple projects
Manifest-published	BSP in its own Git repo, referenced via custom manifest	Organization-wide standard, integrated with Library Manager (power-user)

Project-Local (Simplest)

The BSP lives in the project's bsps/ directory and is committed along with the project source code:

- Self-contained — no external dependencies
- Version controlled by the project's own Git history
- Best for single projects or when BSP changes are project-specific

Shared via Git Repository (Recommended for Teams)

For teams working on multiple products that share the same hardware platform:

1. Commit your custom BSP(s) to a dedicated Git repository
2. Team members clone the repository to a local folder
3. For new projects: In Project Creator, browse to the local folder and select the custom BSP as the target
4. For existing projects: Use Library Manager to add the BSP from the local folder into the project

Important: In both cases, the BSP is COPIED into the project — it is not linked. Once copied, the project-local version is independent from the original Git repository. If you update the BSP in the shared repository, existing projects do not automatically receive those updates; you must manually update each project.

Manifest-Published (Power-User / Organization-Wide)

For large organizations that want BSPs to appear in Library Manager alongside Infineon's official BSPs:

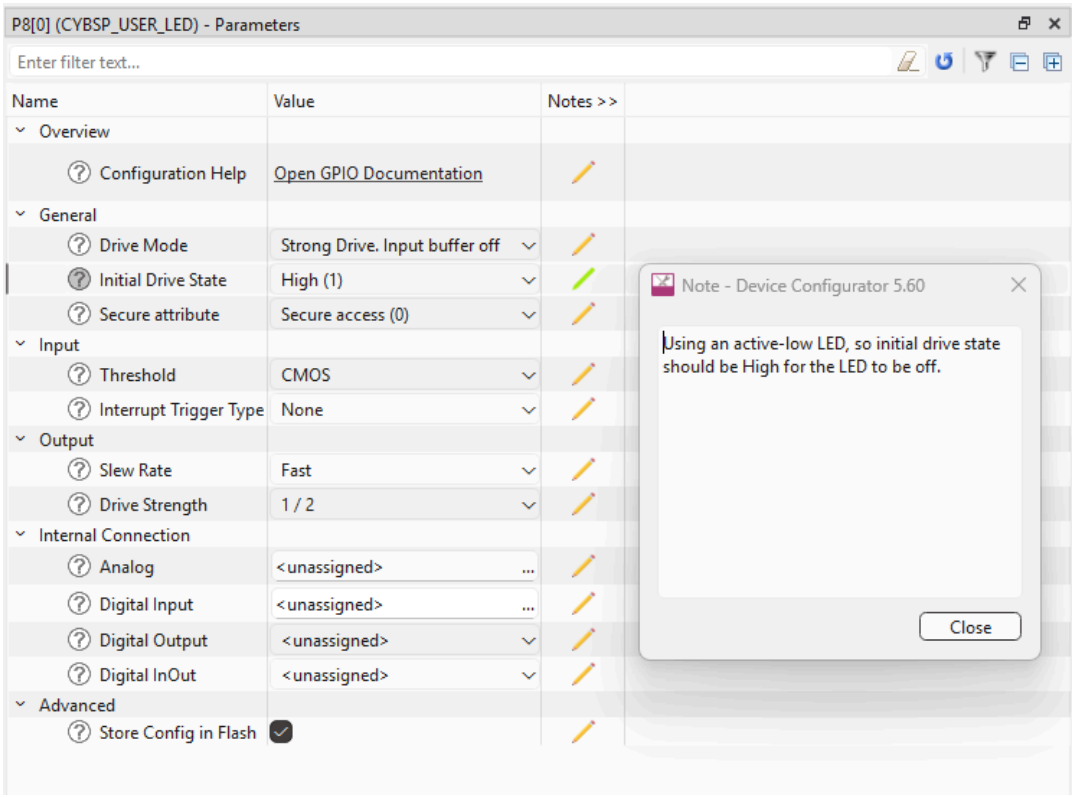
- BSP lives in its own Git repository
- A custom super-manifest references the BSP repository
- The BSP appears in Library Manager for easy selection
- Version it with Git tags for pinning

This approach requires more infrastructure setup but provides the most integrated experience for large teams.

4.5 Best Practices for Custom BSPs

Following these practices will make your custom BSPs more maintainable and reduce troubleshooting time.

Practice	Why
Start from a kit BSP	Ensures correct startup, linker, and configurator files, allows prototyping on Infineon provided board and then adapting to customer hardware
Modify freely — the BSP is yours	BSPs were moved into the project specifically so users can customize them. The original Infineon BSP can always be restored via Library Manager
Use descriptive naming for custom boards	TARGET_MY_PRODUCT_V2 — makes it clear this targets specific hardware
Version your custom BSP	Use Git tags so projects can pin to a known-good BSP version
Document pin changes in Device Configurator	Use the User Notes feature to annotate pin assignments, clock choices, and design rationale directly in design.modus
Test with Device Configurator first	Validate pin/clock config before writing application code



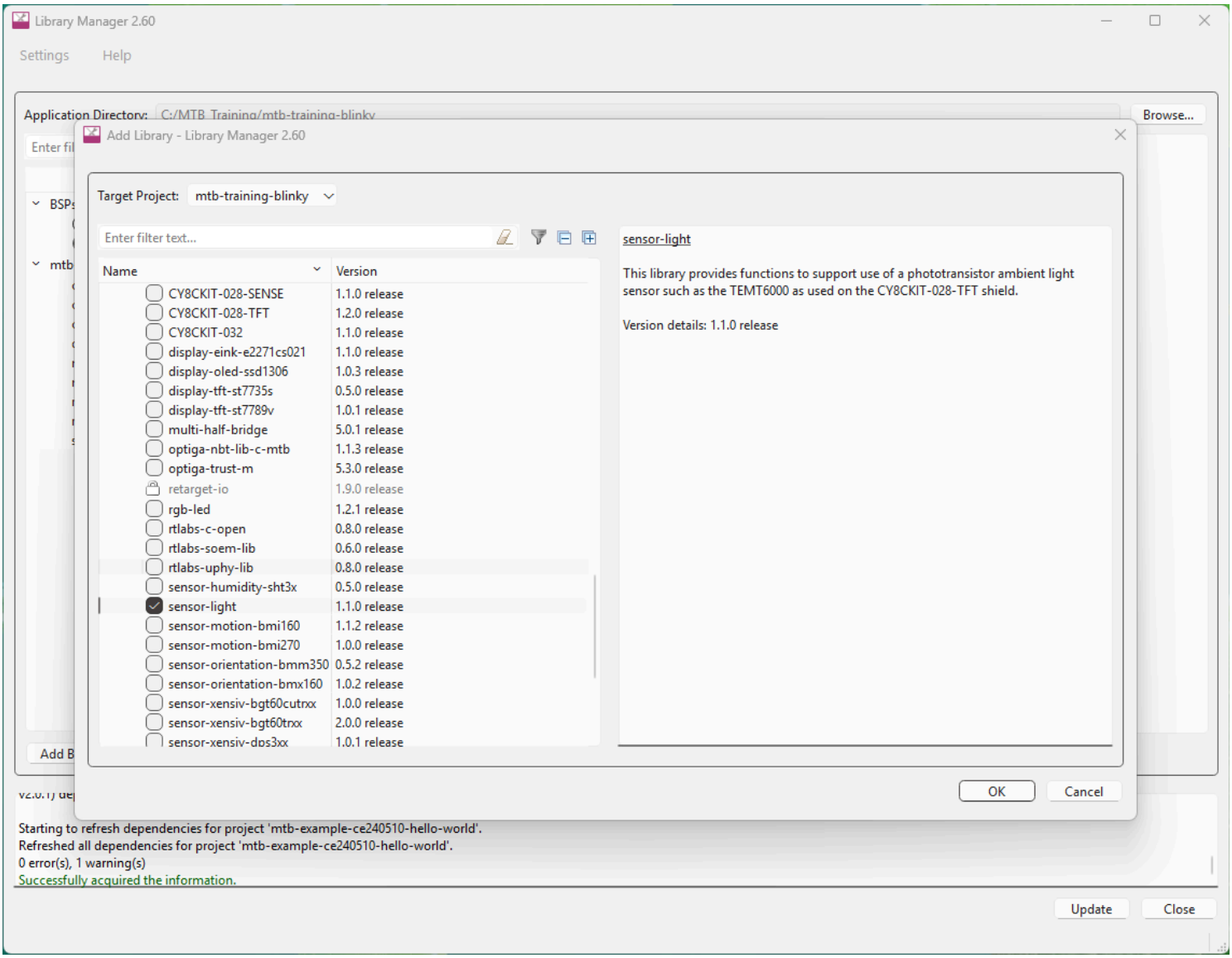
The **User Notes** feature in Device Configurator is particularly valuable for documenting why specific configuration choices were made. When someone revisits the BSP months later (possibly you), these notes explain the reasoning behind clock frequencies, pin assignments, and peripheral configurations.

Chapter 5: Middleware Integration — Library Manager & Makefiles

Middleware libraries extend your application's capabilities beyond what the BSP provides — UART retargeting, RTOS integration, wireless connectivity, sensor drivers, and more. This chapter covers how to manage middleware using Library Manager and the underlying mechanisms that make it work.

5.1 Library Manager Overview

Library Manager is the graphical tool for managing middleware in your ModusToolbox™ project. It provides a visual interface for adding, updating, and removing libraries without manually editing .mtb files.



Library Manager enables you to:

- **View** all middleware currently in your project
- **Add** new middleware libraries from Infineon's catalog
- **Update** existing libraries to newer versions
- **Remove** libraries you no longer need

Launching Library Manager

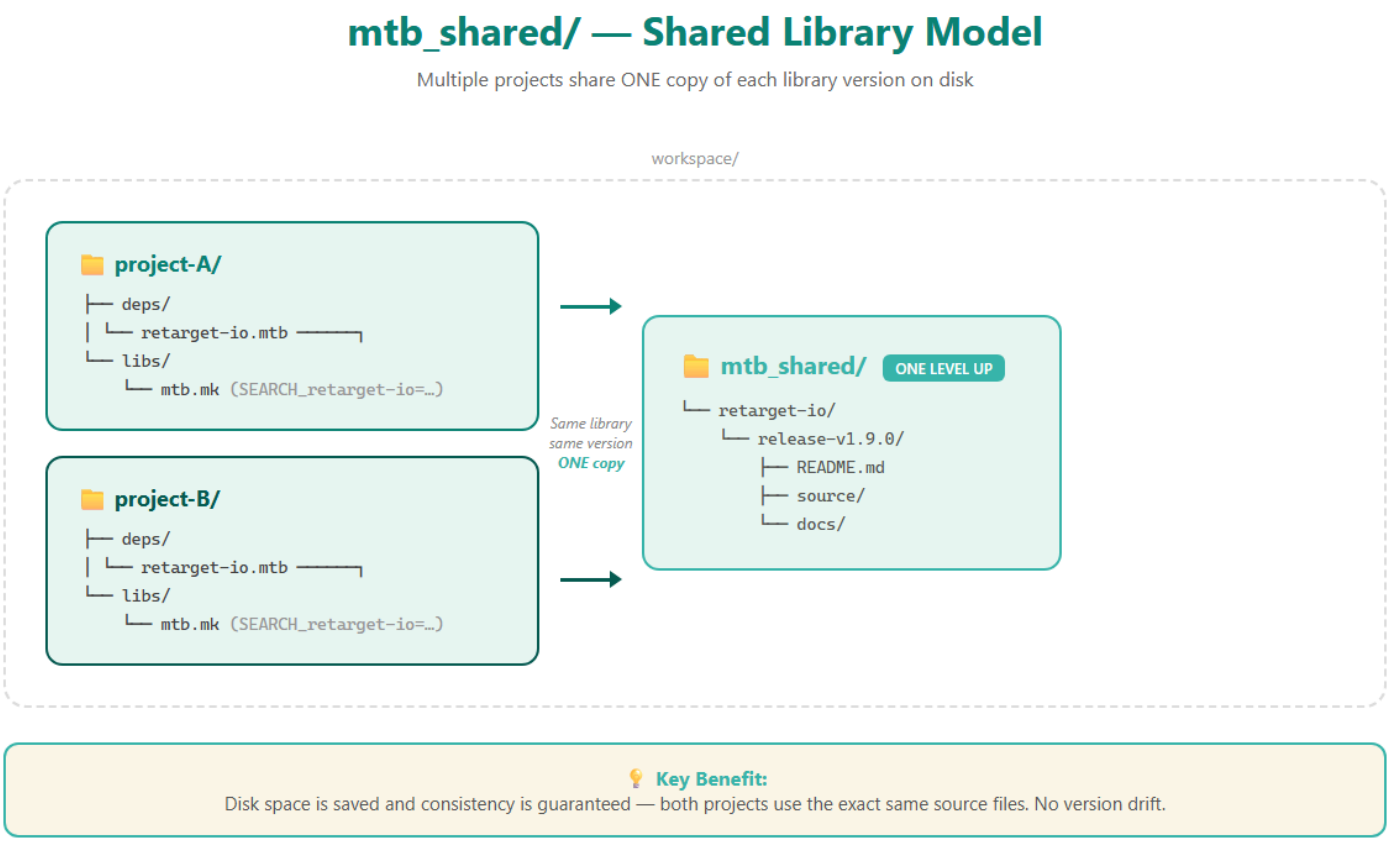
Library Manager can be launched from multiple locations:

- **Eclipse IDE**: Tools → Library Manager (or from the Quick Panel)
- **VS Code**: ModusToolbox Assistant extension → Application tab → Configure Middleware
- **Command line**: `make modlibs` or `make library-manager`

When Library Manager opens, it shows your project's current dependencies and available updates. The interface groups libraries by category and indicates version status.

5.2 The mtb_shared Model

ModusToolbox uses a shared library model to avoid duplicating library source code across projects. Understanding this model is essential for troubleshooting and for understanding how dependency resolution works.



The mtb_shared/ directory (located one level above your project) serves as a workspace-level library cache. When multiple projects reference the same library at the same version, they share a single copy on disk.

Key characteristics of the mtb_shared model:

- **One copy on disk, shared across all projects at that version**: If project-A and project-B both use retarget-io v1.9.0, there is only one copy in mtb_shared/retarget-io/release-v1.9.0/
- **Multiple versions coexist**: If project-A uses v1.8.0 and project-B uses v1.9.0, both versions exist side-by-side in mtb_shared/
- *libs/mtb.mk contains SEARCH_ paths**: Each project's mtb.mk points the build system to the appropriate version in mtb_shared/
- **Pin to release tags in .mtb files for reproducible builds**: Using specific version tags (release-v1.9.0) rather than branch names (main) ensures consistent builds

5.3 Adding Middleware: What Happens Under the Hood

When you add a library using Library Manager, several steps occur automatically:



Library Manager is essentially a graphical interface for managing .mtb files. The actual fetching and resolution is performed by `make getlibs`, which Library Manager invokes when you click "Update."

Note: Users working through the IDE (Project Creator, Library Manager) should rarely need to call `make getlibs` directly — the tools handle it automatically. Direct `make getlibs` invocation is primarily useful when working from the command line only (for example, after a `git clone`) or for troubleshooting.

5.4 Key Makefile Variables

Your project's Makefile contains variables that control the build process. Understanding these variables helps you customize builds and integrate middleware correctly.

```makefile
# ── Target & Toolchain ──
TARGET=APP_KIT_PSC3M6_EVAL      # Active BSP
TOOLCHAIN=GCC_ARM               # Compiler (GCC_ARM, ARM, IAR)
CONFIG=Debug                     # Build configuration

# ── Source & Search Paths ──
SOURCES=                         # Additional source files (rarely needed)
INCLUDES=                        # Additional include paths

# ── Components & Features ──
COMPONENTS=                      # Enable optional BSP/library components
                                 # e.g., COMPONENTS+=FREERTOS
```

```
DEFINES=                        # Preprocessor defines
                                # e.g., DEFINES+=CY_RETARGET_IO_CONVERT_LF_TO_CRLF

# ── Middleware Control ──
DISABLE_COMPONENTS=             # Disable default components
CY_IGNORE=                      # Paths to exclude from build
```

The most frequently modified variables are:

- **TARGET**: Specifies which BSP to use
- **TOOLCHAIN**: Selects the compiler (GCC_ARM, ARM, or IAR)
- **CONFIG**: Debug or Release build
- **COMPONENTS**: Enables optional library features (covered in detail in Section 5.6)
- **DEFINES**: Adds preprocessor definitions used by libraries

5.5 Finding Middleware Documentation (README & docs/)

This is the most important section in Chapter 5.

The number one question FAEs receive after a customer adds a library is: "I added it — now what?" The answer is always the same: **read the library's README.md**.

Location	What You'll Find	How to Access
Library README.md	Quick-start guide, required COMPONENTS and DEFINES, API overview, code snippets	Open mtb_shared/library/version/README.md in your IDE, or view on the library's GitHub repo
Library docs/ folder	Full API reference (HTML), configuration guide, migration notes	Open mtb_shared/library/version/docs/
IDE quick links	Direct access to API documentation for libraries in your project	Eclipse: Quick Panel → Documentation section; VS Code: ModusToolbox Assistant extension → Documentation links
Library GitHub repo	README (same as above), release notes, code examples list, issue tracker	Click the library URL in the .mtb file or find it in Library Manager
Code examples	Working, tested projects that demonstrate the library in context	Links in the library README under "Code Examples" or "Related Resources"

Practical Workflow When Adding a New Library

Follow this sequence every time you add a library:

1. **Add library via Library Manager** → click Update
2. **Open mtb_shared/library/version/README.md** — START HERE
3. **README tells you:**
   - Required COMPONENTS
   - Required DEFINES
   - Required companion libraries
   - Initialization code
4. **For deeper API details** → open docs/api_reference.html
5. **For working examples** → follow Code Example links in README

Example: Adding retarget-io

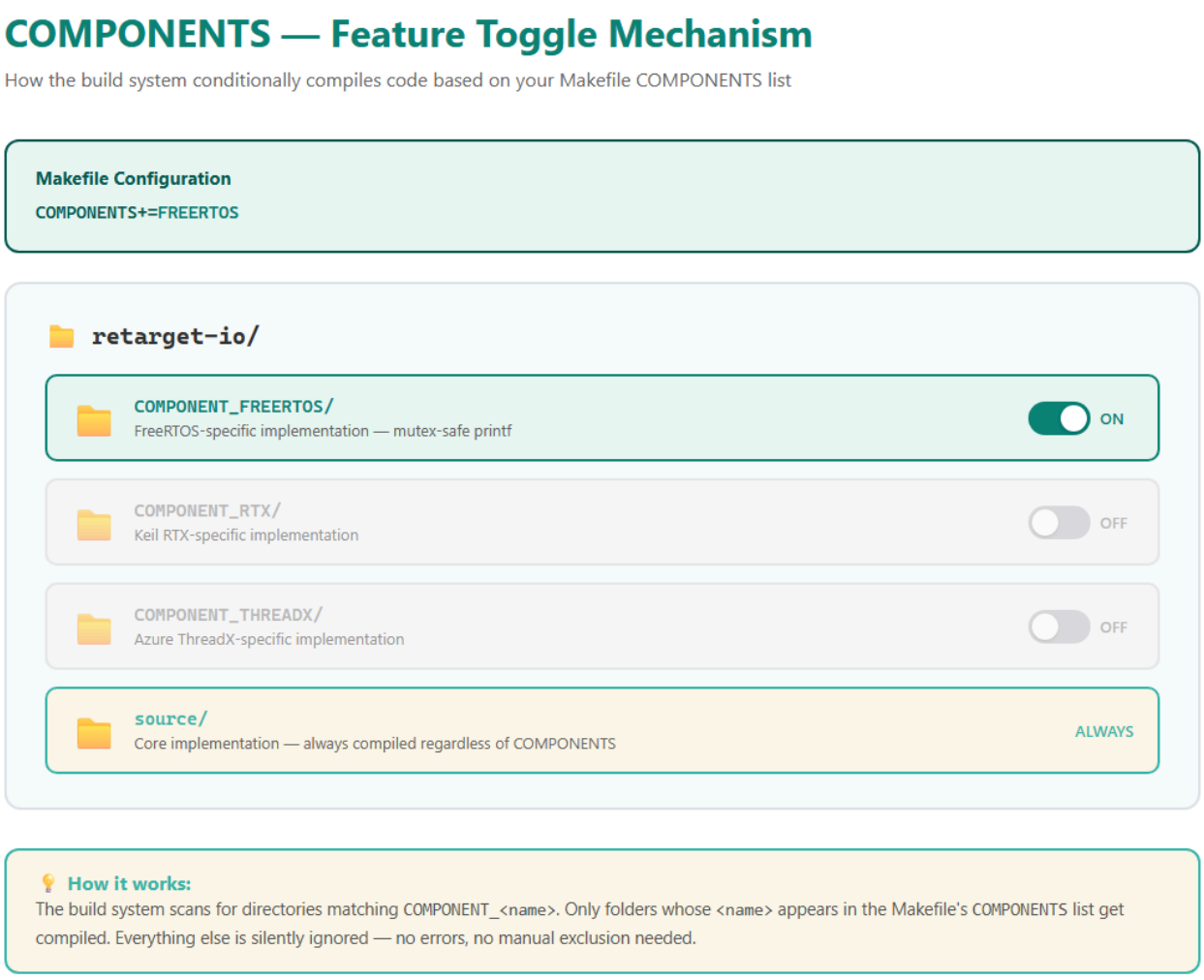
When you add the retarget-io library, its README provides everything you need:

```
mtb_shared/retarget-io/release-v1.9.0/README.md tells you:
  ✓ No COMPONENTS required (works bare-metal and with RTOS)
  ✓ Optional DEFINE: CY_RETARGET_IO_CONVERT_LF_TO_CRLF
  ✓ Call cy_retarget_io_init() before printf
  ✓ Companion: needs HAL (already in BSP)
```

Do not guess at library requirements — the documentation that ships with each library is your authoritative source.

5.6 COMPONENTS: The Feature Toggle System

The COMPONENTS mechanism allows libraries to include or exclude code paths based on your project's configuration. This enables a single library to support multiple use cases without bloating every build with unused code.



How COMPONENTS Work

The build system scans for COMPONENT_* directories and includes only those whose component name appears in your Makefile's COMPONENTS variable. Code in source/ (or any directory without the COMPONENT_ prefix) is always compiled.

Common COMPONENTS Values

Component	Effect
FREERTOS	Enables FreeRTOS-specific code paths in libraries
CM0P_SLEEP	Cortex-M0+ sleep image for dual-core devices (e.g., PSOC 6)

How to Know Which COMPONENTS a Library Needs

Check the library's README.md. Every well-documented Infineon library specifies its required and optional COMPONENTS. If a library requires FREERTOS to function correctly, the README states this explicitly.

To add a component to your project, edit the Makefile:

```makefile
COMPONENTS+=FREERTOS
```

Chapter 6: Introduction to the Upcoming ModusToolbox SDK

> ⚠️ **Disclaimer:** The SDK model described in this chapter is under active development. Specific details — including project structure, tooling behavior, and migration workflows — are subject to change before the official release. The information here reflects the current design intent and is provided so FAEs can understand the direction and prepare accordingly. Refer to official ModusToolbox™ release notes for final specifications.

Infineon is developing an SDK model that will replace the current per-library dependency system with curated, tested, versioned packages. This chapter introduces the SDK concept, compares it to the current model, and explains what FAEs need to know to prepare for the transition.

6.1 The Current Model vs. the SDK Model

The current ModusToolbox™ dependency model gives developers fine-grained control over individual library versions via `.mtb` files. Each project declares its dependencies in a `deps/` directory, and `make getlibs` clones the corresponding library source from GitHub into a shared `mtb_shared/` directory. The upcoming SDK model eliminates this mechanism entirely — bundling tested combinations of BSPs, middleware, code examples, and tools into versioned packages installed locally.

Current Model vs SDK Model

How ModusToolbox™ dependency management is evolving

	📦 Current Model TODAY	🚀 SDK Model FUTURE
Dependencies	Individual .mtb file per library	SDK version bundles BSPs + middleware + tools
Version Management	Each library versioned independently	One SDK version = tested combination of all components
Compatibility	User verifies compatibility between libraries	SDK guarantees a compatible, tested set
Updates	Update libraries individually via Library Manager	Update SDK version → all components update together
Reproducibility	Depends on accurate .mtb version pinning	SDK version = fully reproducible build environment
GitHub Dependency	Required for every project creation and library update	Works fully offline after initial SDK install
Installation	Separate installs for tools and embedded assets	Single SDK install includes everything

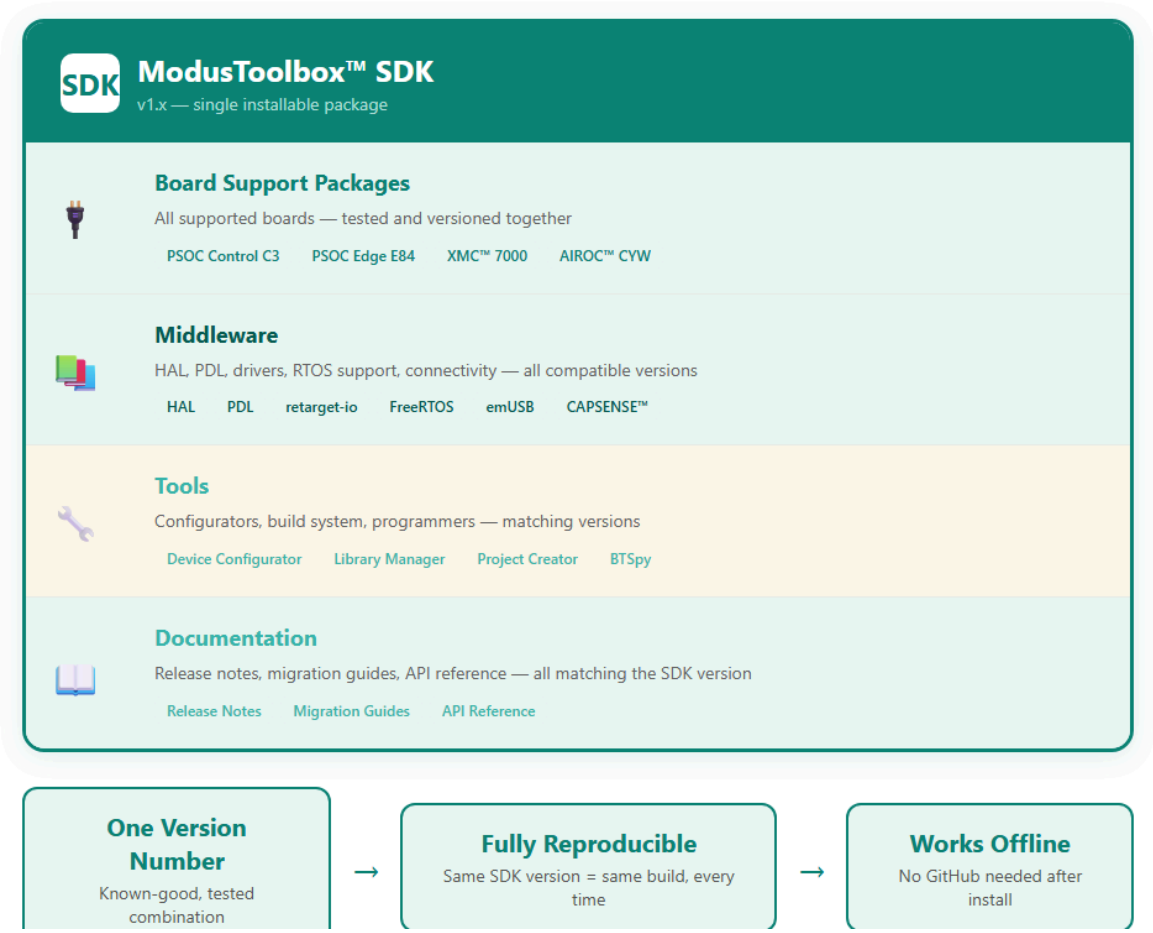
💡 **The transition:**
Understanding the current .mtb model is essential — it's how existing projects work and how you'll maintain them. The SDK model builds on these same concepts but packages them more conveniently.

The current model offers maximum flexibility: you can mix and match library versions to suit your project's needs. However, this flexibility places the burden of compatibility verification on the developer. When updating one library, you must verify it works with all other libraries in your project. The `mtb_shared/` model also means that projects are not self-contained — sharing a project requires the recipient to run `make getlibs` and have GitHub access to reconstruct the full dependency tree.

The SDK model trades some flexibility for simplicity and guaranteed compatibility. When you select an SDK version, you know that all included components have been tested together by Infineon, and the resulting application includes everything it needs to build.

6.2 What the SDK Provides

The SDK bundles multiple components into a single versioned package:



The SDK model provides several benefits:

Predictability: "I'm using SDK v1.2" tells support everything they need to know about your environment. There is no need to enumerate individual library versions.

Tested Together: Infineon validates the full combination before release. You can be confident that HAL version X works correctly with PDL version Y and BSP version Z.

Self-Contained Applications: Middleware source is copied from the SDK into each project's `libs/` directory. Applications include all their dependencies — no external `mtb_shared/` folder required.

Simpler Upgrades: Bump one SDK version instead of updating 15 individual libraries and verifying compatibility. If you have modified a library locally, the upgrade process detects the change and offers to archive your modifications before replacing.

Simplified Installation: A single SDK install includes tools and embedded assets together. After the initial download, the SDK functions fully offline — no GitHub access required for project creation or building.

6.3 Impact on Your Workflow

The SDK model introduces significant structural changes to projects while preserving the core development experience.

Workflow	Current	With SDK
Creating a project	BSP → CE → download manifests → clone from GitHub	BSP → Code Example → SDK → copy from local install
Project structure	`deps/` + `.mtb` files; `libs/` = transitive pointers; `mtb_shared/`	`deps/` and `mtb_shared/` gone; `libs/` = real source; `bsps/` → `bsp/`
Library versions	Add, remove, update individually via Library Manager	Add/remove within SDK; version changes require SDK upgrade
Version control	Commit `.mtb` pointers; `mtb_shared/` not committed	Commit middleware source directly; app is self-contained
Sharing	Recipient must `make getlibs` to reconstruct	App includes everything; SDK needed only for tools
Middleware ownership	Shared in `mtb_shared/`; not typically edited	Local in `libs/`; editable; SDK detects changes on upgrade

Key structural changes in detail:

- `mtb_shared/` **is eliminated.** Middleware source is copied directly from the SDK into each project's `libs/` directory. Libraries live at `libs/retarget-io/` (no version subdirectory).
- `deps/` **directory and** `.mtb` **files are eliminated.** You no longer manage individual dependency pointers.
- `bsps/` **is renamed to** `bsp/` (singular) in SDK-based projects.
- `mtb-props.json` appears at the application root, recording which SDK version the project uses.
- **Library Manager** still manages libraries but within the scope of the SDK. You can add or remove libraries, but changing an individual library's version requires upgrading the SDK. The "Add BSP" function is replaced with "Replace BSP."

What Doesn't Change

The fundamental development workflow remains the same:

- Makefile structure and build commands (`make build` , `make program`)
- Device Configurator workflow
- BSP customization workflow
- COMPONENTS system for feature toggles

The SDK model builds on the same underlying infrastructure. Developers who understand project structure, the build system, and BSP architecture will find the SDK transition straightforward. The concepts covered in Chapters 1–5 of this guide remain directly applicable.

6.4 Preparing for the SDK Transition

As an FAE, here are the key actions to prepare for the SDK:

1. **Understand the current model deeply**: Everything covered in the earlier chapters of this guide remains foundational. The SDK builds on these concepts rather than replacing them.
2. **Watch for SDK release announcements**: Information will come through the usual ModusToolbox™ channels — release notes, documentation updates, and community announcements.
3. **Expect a transition period**: Both models will coexist for some time. Existing projects will continue to work with the current per-library model.
4. **Migration tooling available**: Project Creator will include a guided migration option that converts existing projects to the SDK model. The process is non-destructive — your original project is left untouched, and a new SDK-based project is created alongside it.

5. **Practice good version discipline now**: Understanding dependency management today makes the SDK transition natural. Customers who already understand how their projects are structured will find the migration straightforward.

The SDK does not invalidate the knowledge in this guide — it builds on top of it. Customers who understand project structure, BSP architecture, Makefiles, and COMPONENTS will find the SDK transition straightforward. The key paradigm shifts to communicate: `deps/` and `mtb_shared/` go away, middleware moves into the project's `libs/` directory as real source (not pointers), and library versions are locked to the SDK. But the underlying concepts remain the same — and that is why the foundational knowledge from this training matters.

Appendix A: ModusToolbox Command Reference

This appendix provides a quick reference for the most commonly used ModusToolbox commands.

Make Targets

Command	Description
`make getlibs`	Download and update all dependencies from .mtb files. Clones libraries into mtb_shared/ and generates libs/mtb.mk with SEARCH paths. Safe to run multiple times (idempotent).
`make build`	Compile the project using the configured toolchain and build settings. Produces the binary in build/CONFIG/.
`make program`	Flash the compiled binary to the connected development board. Requires a debug probe connection.
`make clean`	Delete all build artifacts in the build/ directory. Does not affect mtb_shared/ or libs/.
`make modlibs`	Launch Library Manager from the command line. Equivalent to opening Library Manager from the IDE.
`make bsp-assistant`	Launch BSP Assistant from the command line. Opens the BSP Assistant for the current project's BSP.
`make eclipse`	Generate Eclipse project files (.project, .cproject). Run after `make getlibs` to set up IDE integration.
`make vscode`	Generate VS Code project files (.vscode/ directory and .code-workspace). Run after `make getlibs` .
`make ewarm8`	Generate IAR Embedded Workbench project files.
`make uvision5`	Generate Keil µVision project files.

CLI Tools

Tool	Description
`mtbarchive-gui`	Graphical project archive tool. Launch from Eclipse Quick Panel → **ModusToolbox archiving and sharing** or VS Code ModusToolbox™ Assistant.
`mtbarchive` (CLI)	Command-line archive tool. **Not on the PATH by default** — requires the full path: `C:/Users/<user>/ModusToolbox/tools_3.9/mtbarchive/mtbarchive.exe`

Common Usage Patterns

Clone and build a shared project:

```
git clone <project-url>
cd my-project
make getlibs
make build
```

Create a shareable archive (GUI recommended, or CLI with full path):

```
C:/Users/<user>/ModusToolbox/tools_3.9/mtbarchive/mtbarchive.exe -o project_archive.zip -f .
```

Update all dependencies to latest pinned versions:

```
make getlibs
```

Regenerate IDE files after dependency changes:

```
make eclipse    # For Eclipse IDE
make vscode     # For VS Code
```

Appendix B: Glossary

BSP (Board Support Package): A collection of files that configure ModusToolbox for a specific hardware board. Includes pin configurations, clock settings, linker scripts, and board-level macros.

COMPONENTS: A Makefile variable that enables optional features in libraries. Libraries use COMPONENT_* directories to organize code that is only compiled when the corresponding component is enabled.

cybsp_init(): The BSP initialization function that configures hardware according to the settings in design.modus. Called at application startup before using HAL or PDL functions.

DEFINES: A Makefile variable for preprocessor definitions. Used to configure library behavior (e.g., CY_RETARGET_IO_CONVERT_LF_TO_CRLF).

deps/: A project directory containing .mtb files that declare direct dependencies. These files represent your intent and should be committed to Git.

design.modus: The Device Configurator master file that stores pin, clock, and peripheral configuration. A binary XML file that should only be edited through the Device Configurator GUI.

Device Configurator: A visual tool for configuring pins, clocks, and peripherals. Reads and writes design.modus and generates the cycfg_* source files.

DSL (Device Support Library): A single, consolidated, versioned package providing everything a device family needs for software development — PDL, HAL, device utilities, and build recipe make files — released together under one version number. PSC3 family devices (including PSC3M6) use this model; the BSP declares one DSL dependency (e.g., `mtb-dsl-psc3m6`) instead of separately-versioned HAL and PDL libraries. The HAL and PDL API layers themselves are unchanged — only the packaging and versioning are consolidated.

HAL (Hardware Abstraction Layer): A high-level API for hardware peripherals that provides portability across device families. On PSC3 family devices this ships bundled inside the DSL package rather than as an independently-versioned library.

Library Manager: A GUI tool for adding, updating, and removing middleware libraries. Manages .mtb files and invokes make getlibs.

libs/: A project directory containing the resolved dependency tree (transitive .mtb files and generated mtb.mk). Regenerated by make getlibs — never commit to Git.

make getlibs: A make target that resolves all .mtb dependencies, clones libraries into mtb_shared/, and generates libs/mtb.mk with SEARCH paths.

.mtb file: A text file declaring a library dependency. Contains a Git URL, version tag, and local path hint. The .mtb format replaced the older .lib format.

mtb_shared/: A workspace-level directory (one level above projects) that stores cloned library source code. Shared across all projects at the same version. Regenerated by make getlibs — never commit to Git.

mtb.mk: The generated file in libs/ that contains SEARCH_* variables pointing the build system to library locations in mtb_shared/.

PDL (Peripheral Driver Library): A low-level driver library providing direct access to hardware registers and peripheral functionality. On PSC3 family devices this ships bundled inside the DSL package rather than as an independently-versioned library.

*SEARCH_ variables**: Makefile variables (generated in libs/mtb.mk) that tell the build system where to find library source files.

TARGET: A Makefile variable specifying which BSP to use. Corresponds to a TARGET_* directory in bsps/.

Appendix C: Resources

Official Documentation

Resource	URL
ModusToolbox Documentation Hub	https://documentation.infineon.com/modustoolbox/
ModusToolbox User Guide	Available from the Documentation Hub
Device Configurator User Guide	Available from the Documentation Hub
Library Manager User Guide	Available from the Documentation Hub

Code Examples

Resource	URL
Code Examples on GitHub	https://github.com/Infineon (search for mtb-example-)
PSOC Control C3 Examples	Search "mtb-example-psc3" on GitHub

Community & Support

Resource	URL
Infineon Developer Community	https://community.infineon.com
ModusToolbox Forums	Available through the Developer Community

Application Notes

Document	Description
AN235691	ModusToolbox & Friends — Overview of the ModusToolbox ecosystem

Document	Description
Device-specific app notes	Available through the Documentation Hub

---

End of Document

Document Version: 2.0
Last Updated: October 2026
ModusToolbox Version: 3.9
Target Platform: PSOC Control C3 (KIT_PSC3M6_EVAL)